US012417334B2

(12) **United States Patent**
Yang et al.

(10) **Patent No.: US 12,417,334 B2**
(45) **Date of Patent: Sep. 16, 2025**

(54) **LITHOGRAPHY SIMULATION USING A NEURAL NETWORK**

(71) Applicant: **NVIDIA Corporation**, Santa Clara, CA (US)

(72) Inventors: **Haoyu Yang**, Cedar Park, TX (US); **Haoxing Ren**, Austin, TX (US); **Zongyi Li**, Alhambra, CA (US)

(73) Assignee: **NVIDIA Corporation**, Santa Clara, CA (US)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 697 days.

(21) Appl. No.: **17/738,174**

(22) Filed: **May 6, 2022**

(65) **Prior Publication Data**

US 2023/0153510 A1 May 18, 2023

Related U.S. Application Data

(60) Provisional application No. 63/264,278, filed on Nov. 18, 2021.

(51) **Int. Cl.**
G06F 30/398 (2020.01)
G06F 17/14 (2006.01)
(Continued)

(52) **U.S. Cl.**
CPC ***G06F 30/398*** (2020.01); ***G06F 17/142*** (2013.01); ***G06F 30/27*** (2020.01); *G06F 2119/18* (2020.01)

(58) **Field of Classification Search**
CPC G06F 3/011; G06F 3/017; G06F 3/016; G06F 30/398; G06F 18/214;
(Continued)

(56) **References Cited**

U.S. PATENT DOCUMENTS

2017/0323481 A1* 11/2017 Tran H04N 23/611
2020/0380362 A1* 12/2020 Cao G06N 3/04
2021/0286270 A1* 9/2021 Middlebrooks G06N 3/047

OTHER PUBLICATIONS

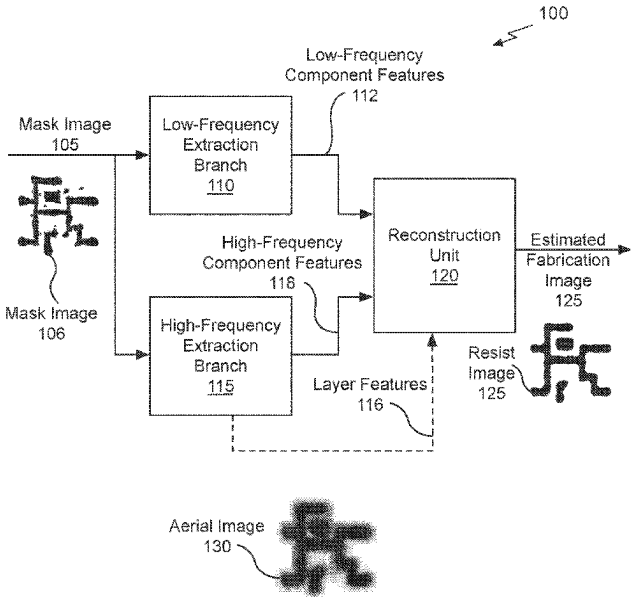
Ma, X., et al., "Computational Lithography," John Wiley & Sons, 2011, vol. 77 (CH 2).
(Continued)

Primary Examiner — Binh C Tat
(74) *Attorney, Agent, or Firm* — Leydig, Voit & Mayer, Ltd.

(57) **ABSTRACT**

As integrated circuit geometries have shrunk, lithography simulation has developed to ensure that the masks used to fabricate the circuits satisfy the chip yield and fabrication turnaround time targets. To manufacture an integrated circuit (chip), an initial layout for the integrated circuit design is processed to compute a wafer image (e.g., resist material "printed" on the wafer using photomasks). Lithography simulation processes the initial layout according to optical physics to compute an estimated wafer image without actually constructing the physical masks or consuming any wafer fabrication resources and may be used to confirm manufacturability of the design layout before it is fabricated. Performing lithography simulation using a dual-band neural network produces accurate results efficiently. Dual-band refers to a dual frequency band processing whereby the input layout (mask image) is separately processed by both a first and second branch to extract low-frequency (global) features and high-frequency (local) features, respectively.

20 Claims, 12 Drawing Sheets



(51) **Int. Cl.**
G06F 30/27 (2020.01)
G06F 119/18 (2020.01)

(58) **Field of Classification Search**
CPC G06F 17/142; G06F 18/2413; G06F 18/24133; G06F 18/2431; G06F 2119/18; G06F 30/27; G03F 1/36; G03F 7/70675; G03F 7/70633; G03F 7/706841; G06N 3/08; G06N 3/0464; G06N 20/00; G06N 3/045
USPC 716/50–56
See application file for complete search history.

(56) **References Cited**

OTHER PUBLICATIONS

Yu, P., et al., "True process variation aware optical proximity correction with variational lithography modeling and model calibration," in Proc. DAC, 2006, pp. 785-790.
Matsunawa, T., et al., "Optical proximity correction with hierarchical bayes model," in Proc. SPIE, vol. 9426, 2015.
Ye, W., et al., "LithoGAN: End-to-end lithography modeling with generative adversarial networks," in Proc. DAC, 2019, pp. 107:1-107:6.
Lin, Y., et al., "Data efficient lithography modeling with transfer learning and active data selection," IEEE TCAD, 2019.
Chen, G., et al., "DAMO: Deep agile mask optimization for full chip scale," in Proc. ICCAD, 2020.
Ye, W., et al., "Tempo: Fast mask topography effect modeling with deep learning," in Proc. ISPD, 2020, pp. 127-134.
Shim, S., et al., "Machine learning-based 3D resist model," in Proc. SPIE, vol. 10147, 2017. (Abstract).
Adam, K., et al., "Using machine learning in the physical modeling of lithographic processes," in Proc. SPIE, vol. 10962, 2019. (Abstract).
Yu, F., et al., "Multi-scale context aggregation by diluted convolutions," 2016.
Yang, H., et al., "Layout hotspot detection with feature tensor generation and deep biased learning," IEEE TCAD, vol. 38, No. 6, pp. 1175-1187, 2019.
Li, Z., et al., "Fourier neural operator for parametric partial differential equations," in Proc. ICLR, 2021.
Gao, J.R., et al., "MOSAIC: Mask optimizing solution with process window aware inverse correction," in Proc. DAC, 2014, pp. 52:1-52:6.
Wang, H., et al., "High-frequency component helps explain the generalization of convolutional neural networks," in Proc. CVPR, 2020, pp. 8684-8694.
Simonyan, K., et al., "Very deep convolutional networks for large-scale image recognition," arXiv preprint arXiv:1409.1556, 2014.
Yang, H., et al., "GAN-OPC: Mask optimization with lithography-guided generative adversarial nets," IEEE TCAD, 2020.
Ronneberger, O., et al., "U-net: Convolutional networks for biomedical segmentation," in Proc. MICCAI, 2015, pp. 234-241.
"ITRS," http://www.itrs2.net.

* cited by examiner

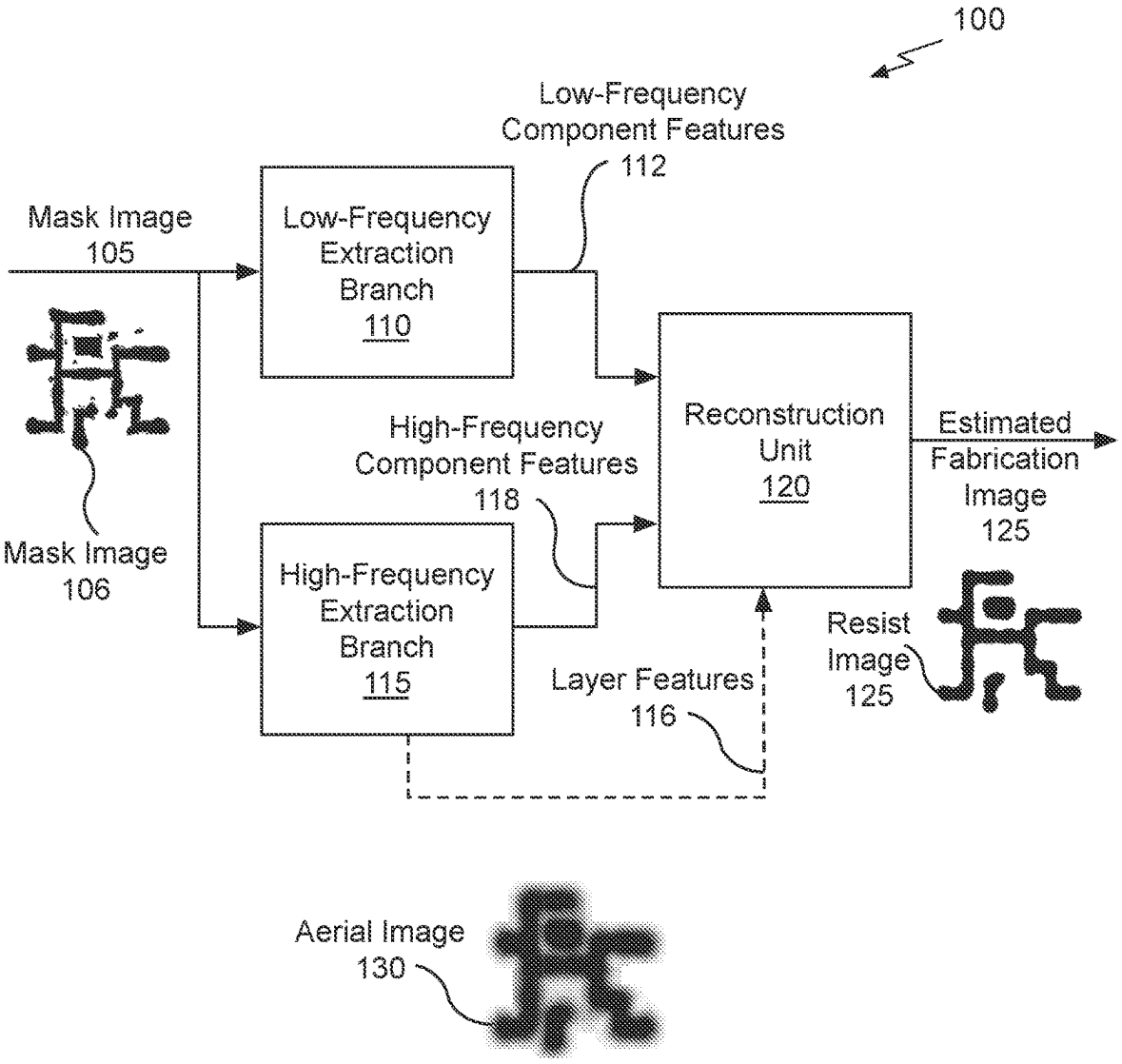

100
Low-Frequency Component Features 112
Mask Image 105
Low-Frequency Extraction Branch 110
Reconstruction Unit 120
Estimated Fabrication Image 125
High-Frequency Component Features 118
Mask Image 106
High-Frequency Extraction Branch 115
Resist Image 125
Layer Features 116
Aerial Image 130

Fig. 1A

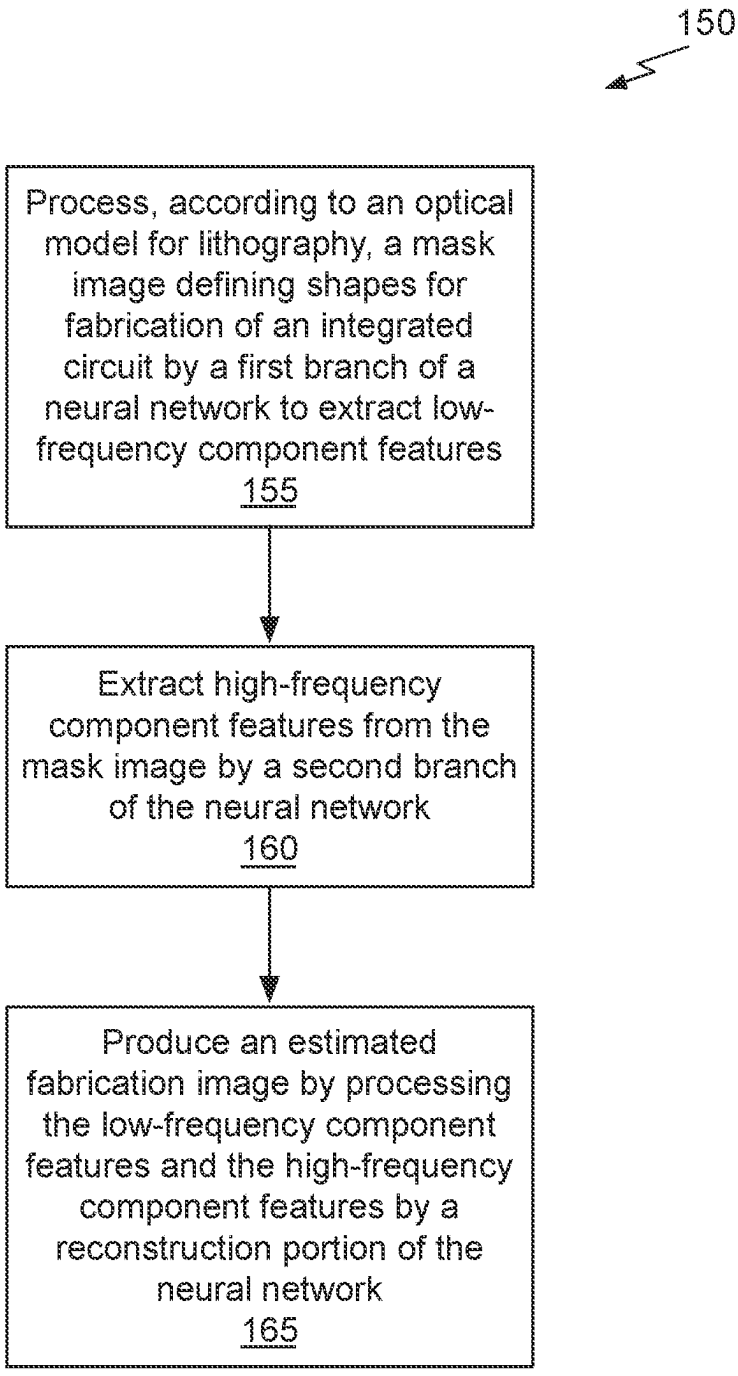

150
Process, according to an optical model for lithography, a mask image defining shapes for fabrication of an integrated circuit by a first branch of a neural network to extract low-frequency component features
155
Extract high-frequency component features from the mask image by a second branch of the neural network
160
Produce an estimated fabrication image by processing the low-frequency component features and the high-frequency component features by a reconstruction portion of the neural network
165

Fig. 1B

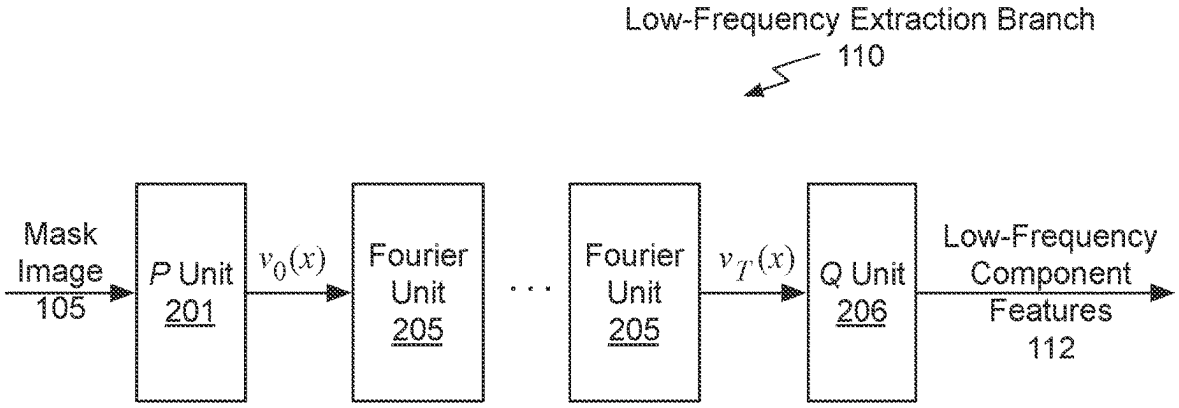

Low-Frequency Extraction Branch
110
Mask Image 105
P Unit 201
$v_0(x)$
Fourier Unit 205
...
Fourier Unit 205
$v_T(x)$
Q Unit 206
Low-Frequency Component Features 112

Fig. 2A

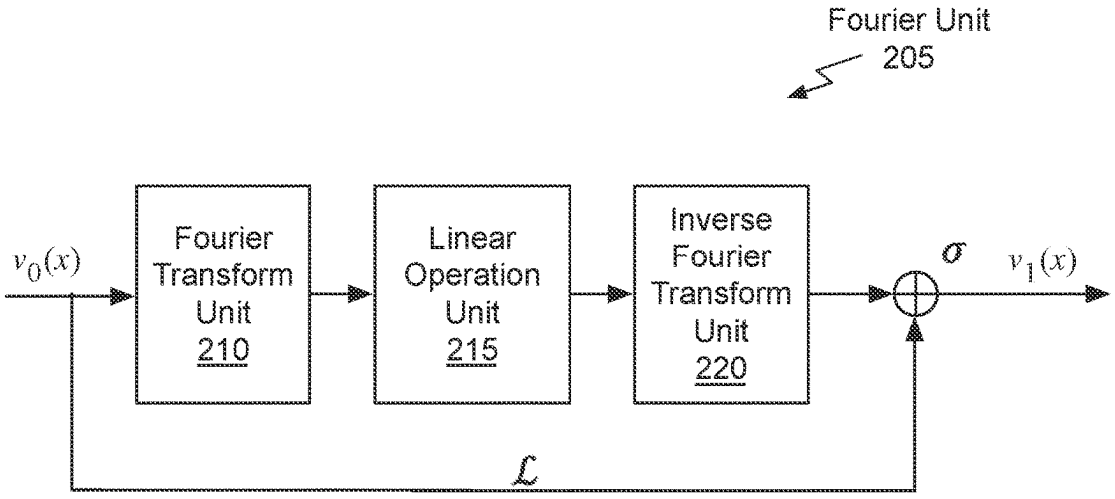

Fourier Unit 205
$v_0(x)$
Fourier Transform Unit 210
Linear Operation Unit 215
Inverse Fourier Transform Unit 220
σ
$v_1(x)$
$\mathcal{L}$

Fig. 2B

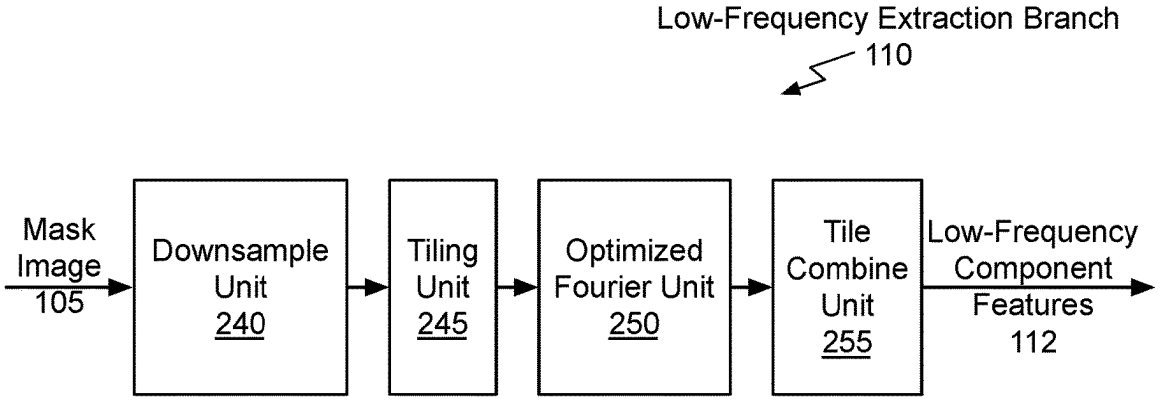

Low-Frequency Extraction Branch
110
Mask Image 105
Downsample Unit 240
Tiling Unit 245
Optimized Fourier Unit 250
Tile Combine Unit 255
Low-Frequency Component Features 112

Fig. 2C

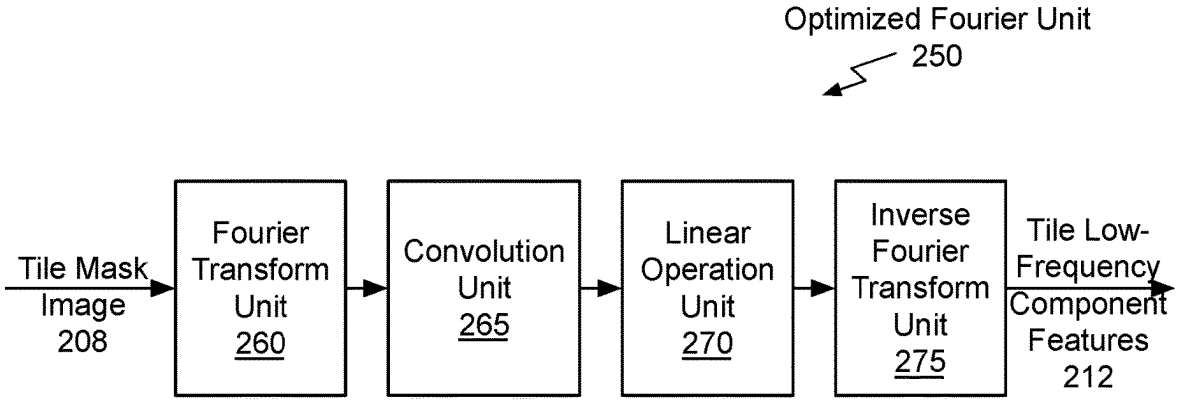

Optimized Fourier Unit
250
Tile Mask Image 208
Fourier Transform Unit 260
Convolution Unit 265
Linear Operation Unit 270
Inverse Fourier Transform Unit 275
Tile Low-Frequency Component Features 212

Fig. 2D

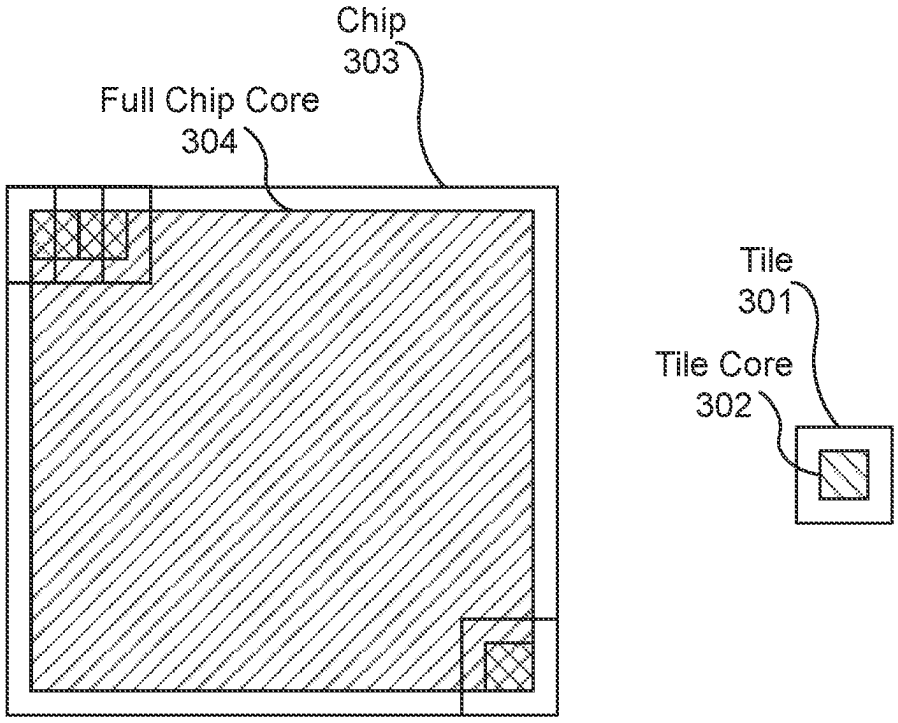


Fig. 3A

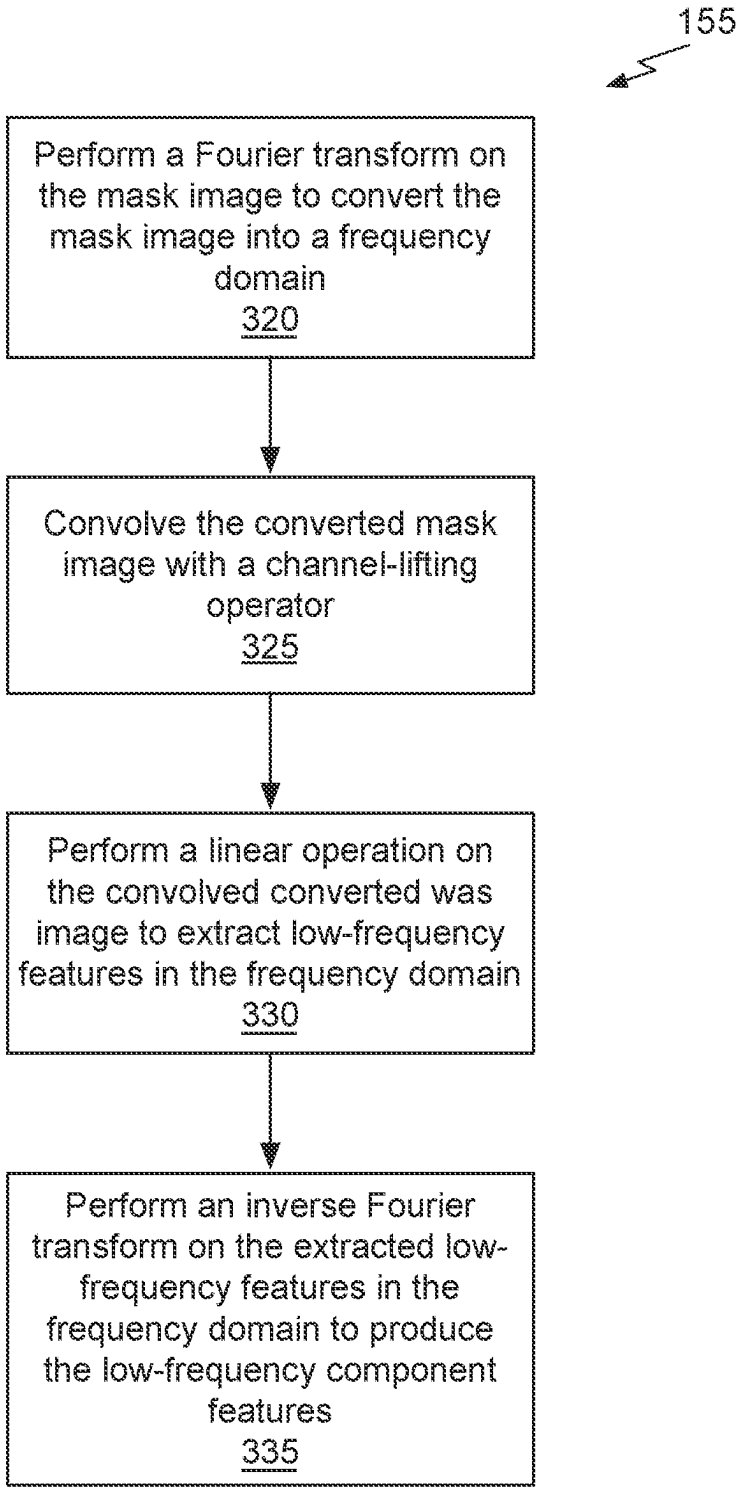

155
Perform a Fourier transform on the mask image to convert the mask image into a frequency domain
320
Convolve the converted mask image with a channel-lifting operator
325
Perform a linear operation on the convolved converted was image to extract low-frequency features in the frequency domain
330
Perform an inverse Fourier transform on the extracted low-frequency features in the frequency domain to produce the low-frequency component features
335

Fig. 3B

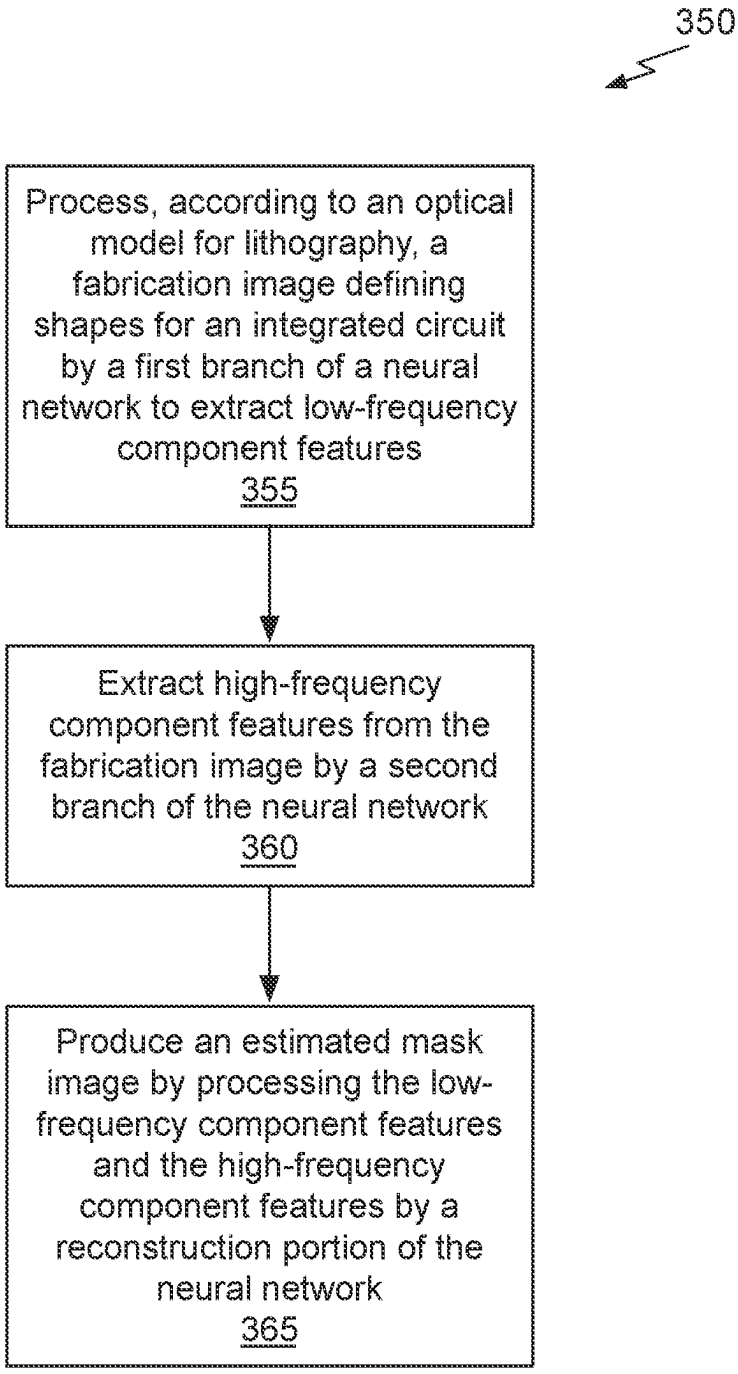

350
Process, according to an optical model for lithography, a fabrication image defining shapes for an integrated circuit by a first branch of a neural network to extract low-frequency component features
355
Extract high-frequency component features from the fabrication image by a second branch of the neural network
360
Produce an estimated mask image by processing the low-frequency component features and the high-frequency component features by a reconstruction portion of the neural network
365

Fig. 3C

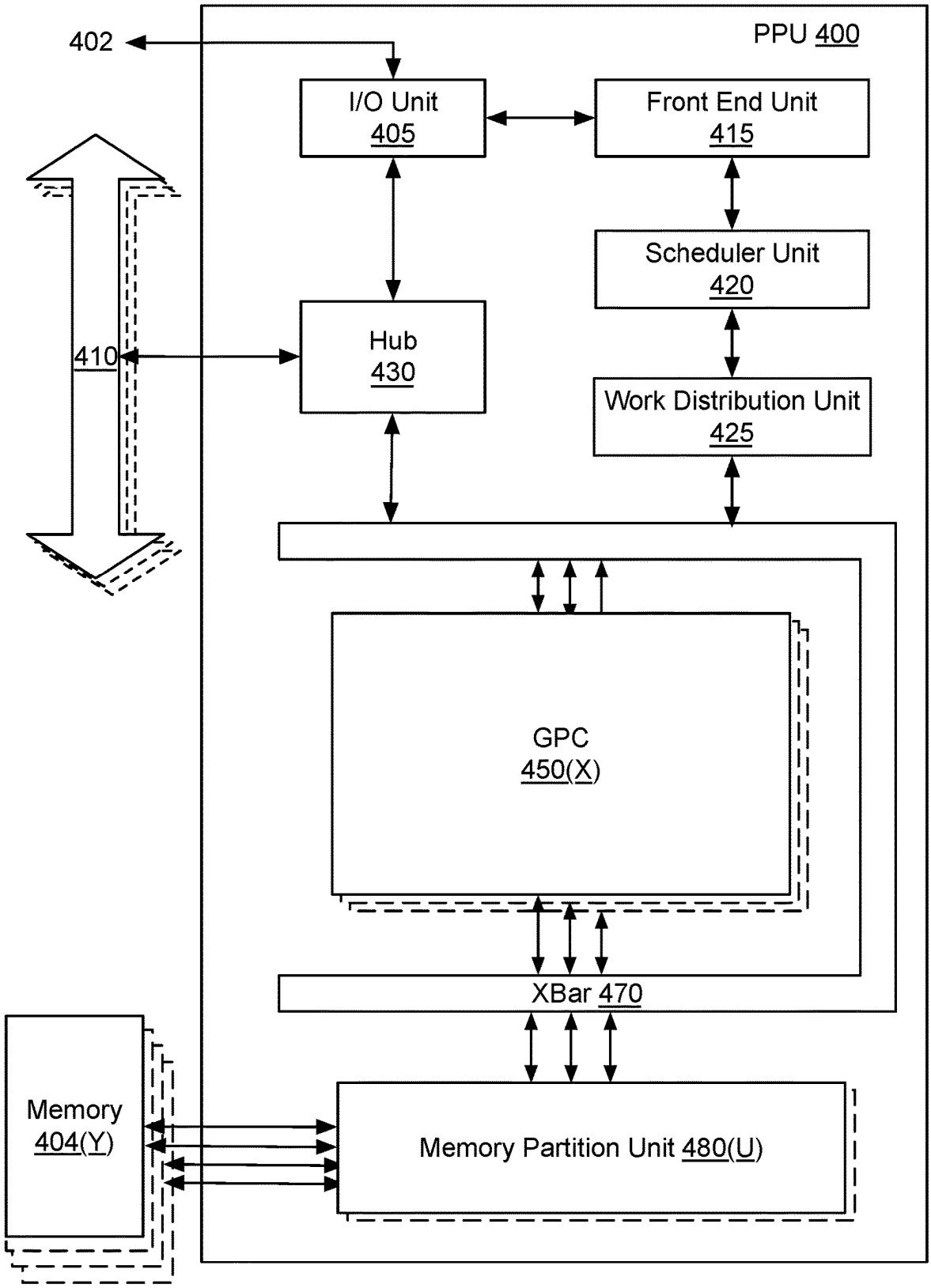

402
PPU 400
I/O Unit
405
Front End Unit
415
Scheduler Unit
420
410
Hub
430
Work Distribution Unit
425
GPC
450(X)
XBar 470
Memory
404(Y)
Memory Partition Unit 480(U)

Fig. 4

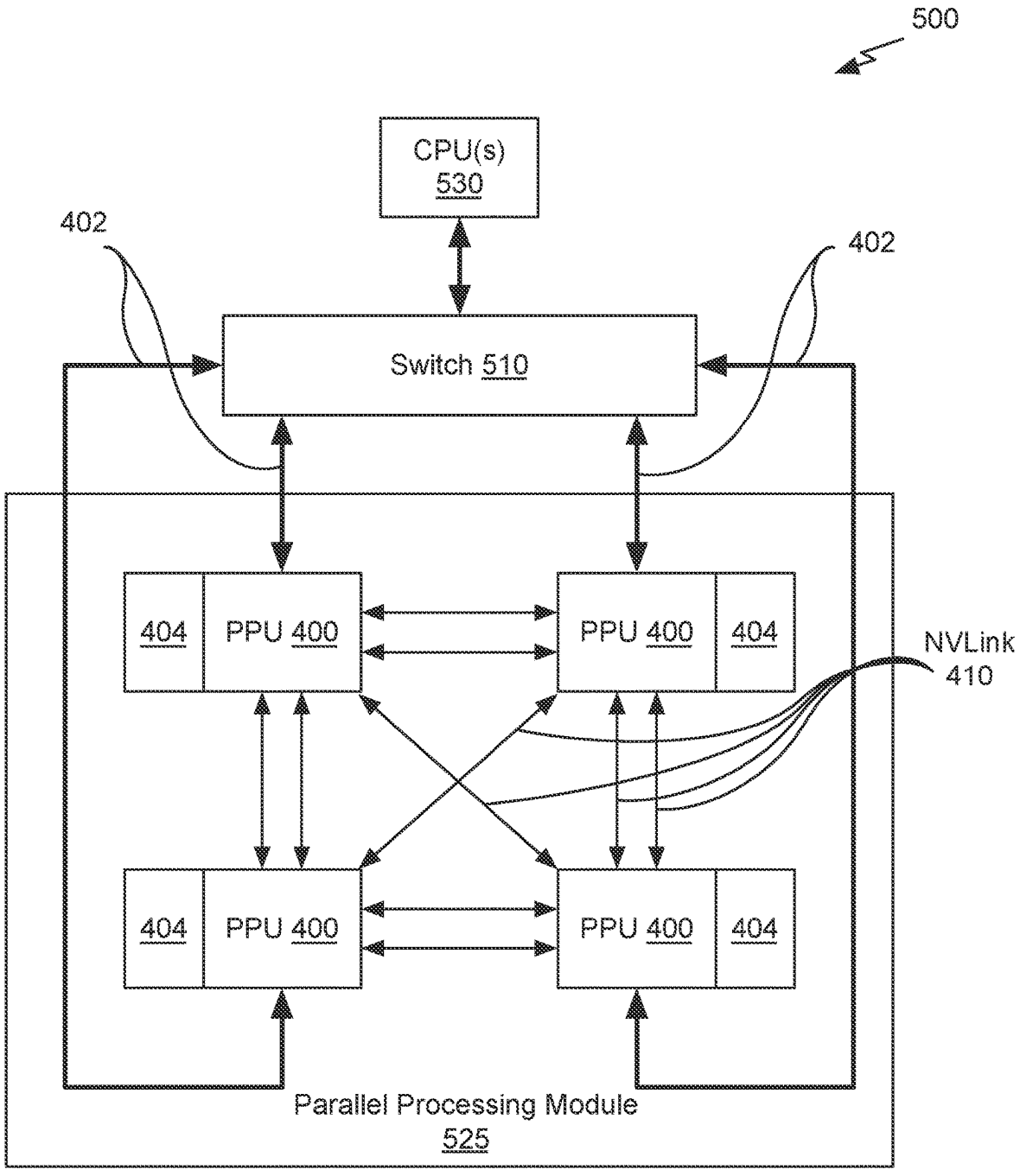

500
CPU(s)
530
402
402
Switch 510
404
PPU 400
PPU 400
404
NVLink
410
404
PPU 400
PPU 400
404
Parallel Processing Module
525

Fig. 5A

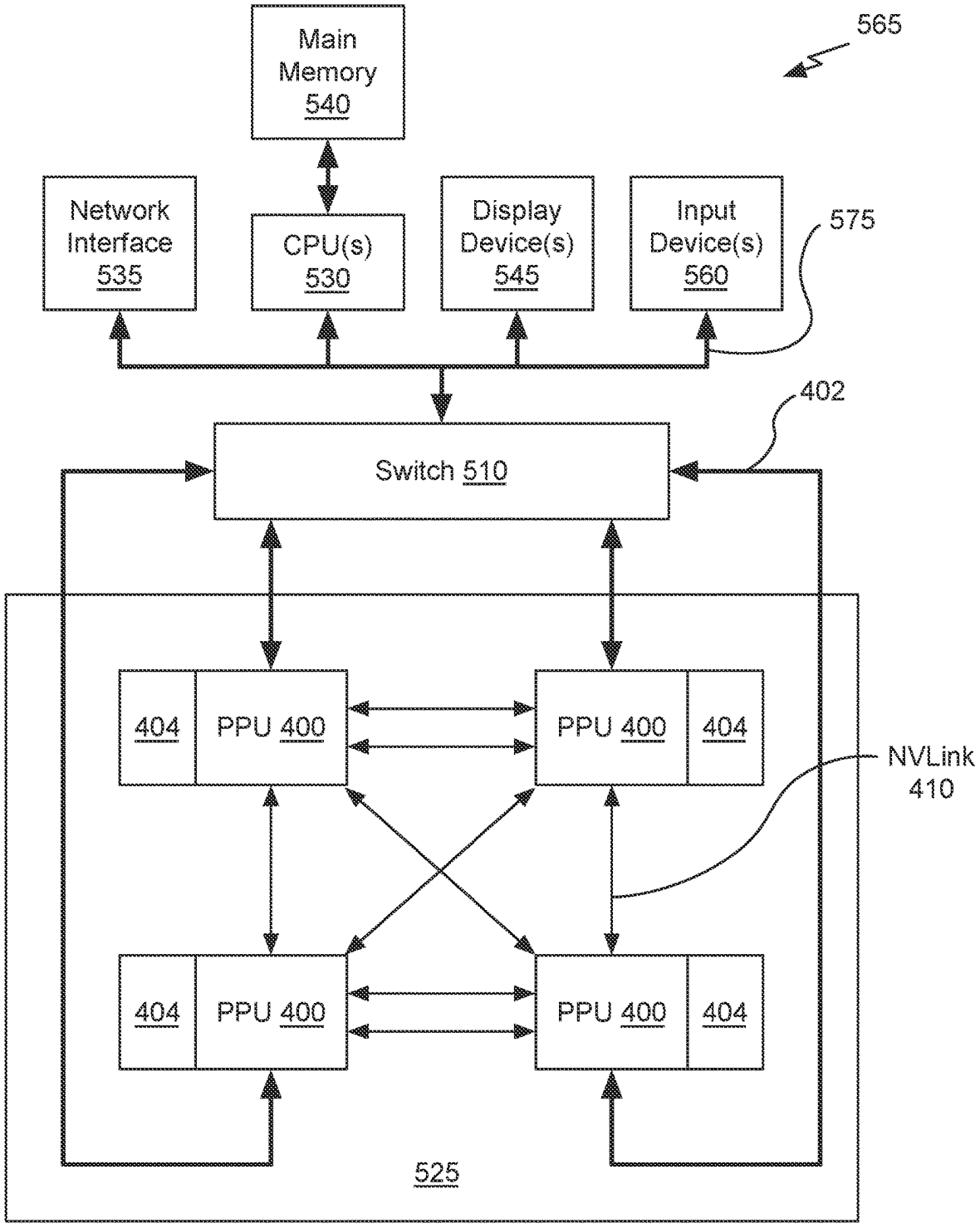

565
Main Memory 540
Network Interface 535
CPU(s) 530
Display Device(s) 545
Input Device(s) 560
575
402
Switch 510
404
PPU 400
PPU 400
404
NVLink 410
404
PPU 400
PPU 400
404
525

Fig. 5B

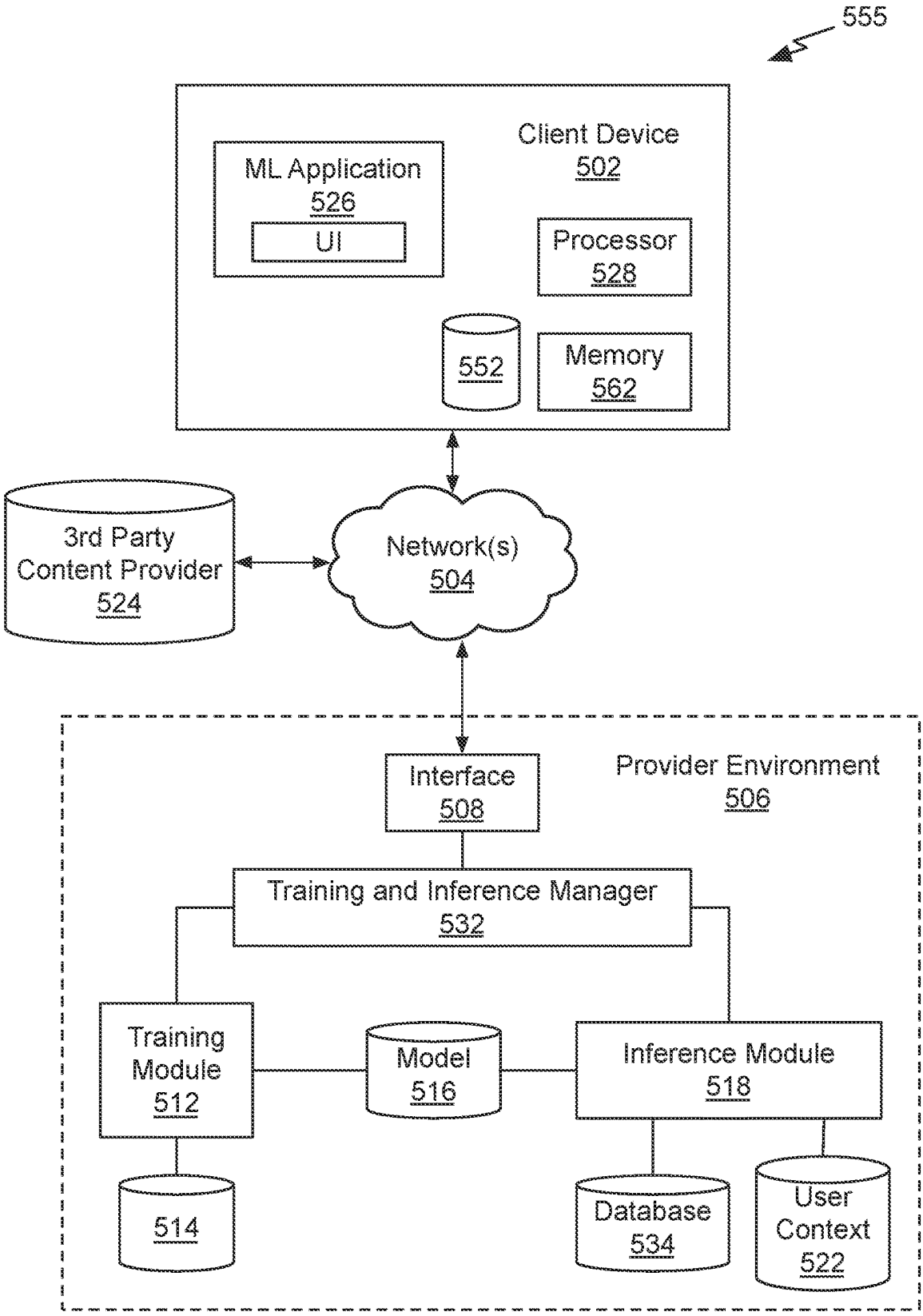

555
Client Device
502
ML Application
526
UI
Processor
528
552
Memory
562
3rd Party
Content Provider
524
Network(s)
504
Provider Environment
506
Interface
508
Training and Inference Manager
532
Training
Module
512
Model
516
Inference Module
518
514
Database
534
User
Context
522

Fig. 5C

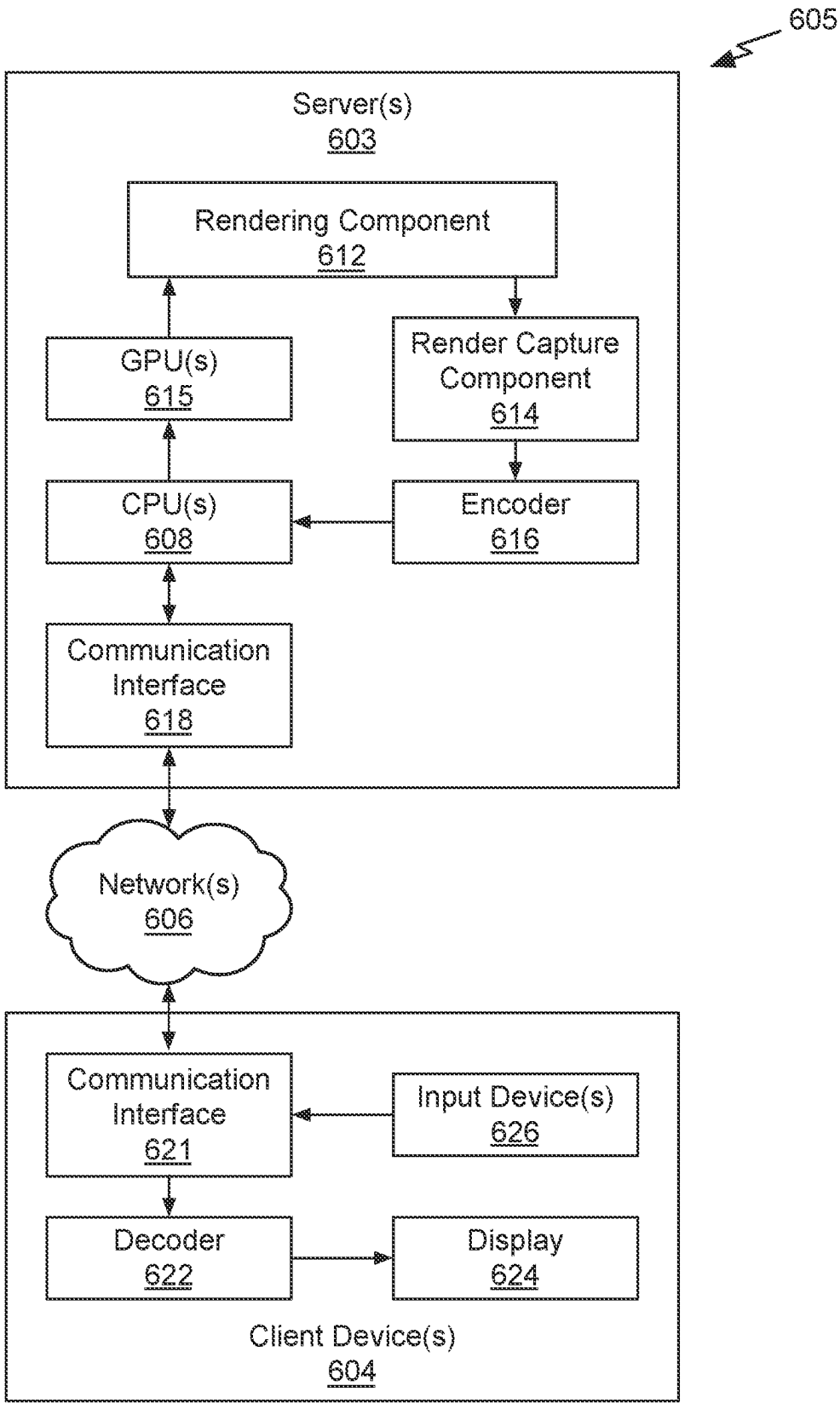

605
Server(s)
603
Rendering Component
612
GPU(s)
615
Render Capture Component
614
CPU(s)
608
Encoder
616
Communication Interface
618
Network(s)
606
Communication Interface
621
Input Device(s)
626
Decoder
622
Display
624
Client Device(s)
604

Fig. 6

LITHOGRAPHY SIMULATION USING A NEURAL NETWORK

CLAIM OF PRIORITY

This application claims the benefit of U.S. Provisional Application No. 63/264,278 titled "LITHOGRAPHY SIMULATION USING A NEURAL NETWORK," filed Nov. 18, 2021, the entire contents of which is incorporated herein by reference.

BACKGROUND

Lithography simulation is a critical step in very large-scale integration (VLSI) design and optimization for manufacturability of integrated circuits. The lithography simulation generates estimated resist contour shapes that will result from the lithography procedure without completing the actual manufacturing process. Conventional solutions require rigorous simulation that is time consuming when seeking high accuracy, even equipped with various approximation techniques. Recently, there are several attempts using a machine learning model for fast lithography modeling. LithoGAN uses conditional generative adversarial nets to predict a resist image directly from mask patterns. LithoGAN has limitations that it only supports single contact prediction and must assume the contact is in the center of input tiles. A contact provides an electrical connection to a transistor source, gate, or drain. The deep lithography simulator (DLS) is the state-of-the-art solution that applies a carefully designed convolutional neural network for contour generation. DLS requires the input of sub-resolution assist features (SRAF) and optical proximity correction (OPC) in different image channels to provide locations of all vias to the neural network. A via provides an electrical connection between different metal layers. Ideally, a lithography simulator should be able to identify all shapes, including contacts and vias, automatically. Although DLS can support multiple shape contour generation, the resolution is limited to 4 nm^2/pixel. Due to the settings of existing layouts, DLS may not be able to perform many critical lithography simulator-backboned tasks. There is a need for addressing these issues and/or other issues associated with the prior art.

SUMMARY

Embodiments of the present disclosure relate to lithography simulation using a neural network. Systems and methods are disclosed that model lithography using an optical compliant dual-band neural network system. Conventional solutions for highly accurate lithography simulation with rigorous models are computationally expensive and slow, even when equipped with various approximation techniques. Recently, machine learning has provided alternative solutions for lithography modeling tasks such as coarse-grained edge placement error regression and complete contour prediction. However, the impact of these learning-based methods has been limited due to restrictive usage scenarios or low modeling accuracy.

In contrast to conventional systems, the dual-band neural network design is compliant with the optical physics underlying lithography. In particular, the dual-band neural network enables via/metal layer contour simulation at 1 nm^2/pixel resolution with any tile size. Compared to conventional machine learning based solutions, experimental results show that the dual-band neural network can be trained faster and offers improvements in efficiency and image quality. The dual-band neural network includes of two perception paths or branches, where a first branch extracts low-frequency global information and a second branch extracts high-frequency local information. In an embodiment, the first branch includes an optimized Fourier unit that significantly reduces computation compared to a Fourier layer within a Fourier neural operator (FNO) while attaining necessary global information to analyze lithography behavior. In an embodiment, the second branch is backboned with convolution layers to capture mask image details and compensate for information loss in the first branch.

Systems and methods are disclosed that perform lithography simulation using a neural network. In an embodiment, a mask image defining shapes for fabrication of an integrated circuit is processed by a first branch of a neural network according to an optical model for lithography to extract low-frequency component features. High-frequency component features are extracted from the mask image by a second branch of the neural network and an estimated fabrication image is produced by processing the low-frequency component features and the high-frequency component features by a reconstruction portion of the neural network.

Systems and methods are disclosed that estimate a mask image from a fabrication image. In an embodiment, a fabrication image defining patterns for an integrated circuit is processed, according to an optical model for lithography, by a first branch of a neural network to extract low-frequency component features. High-frequency component features are extracted from the fabrication image by a second branch of the neural network and an estimated mask image is produced by processing the low-frequency component features and the high-frequency component features by a reconstruction portion of the neural network.

BRIEF DESCRIPTION OF THE DRAWINGS

The present systems and methods for lithography simulation using a neural network are described in detail below with reference to the attached drawing figures, wherein:

FIG. **1**A illustrates a block diagram of a dual-band neural network lithography simulation system, in accordance with an embodiment.

FIG. **1**B illustrates a flowchart of a method suitable for use in implementing some embodiments of the present disclosure.

FIG. **2**A illustrates a block diagram of a low-frequency extraction branch suitable for use in implementing some embodiments of the present disclosure.

FIG. **2**B illustrates a block diagram of a Fourier unit suitable for use in implementing some embodiments of the present disclosure.

FIG. **2**C illustrates another block diagram of a low-frequency extraction branch of the system shown in FIG. **1**A suitable for use in implementing some embodiments of the present disclosure.

FIG. **2**D illustrates a block diagram of an optimized Fourier unit suitable for use in implementing some embodiments of the present disclosure.

FIG. **3**A illustrates a conceptual diagram of a tiling a full chip core for processing by the low-frequency extraction branch, in accordance with an embodiment.

FIG. **3**B illustrates a flowchart of a method for performing a step of the method shown in FIG. **1**B, in accordance with an embodiment.

FIG. **3**C illustrates a flowchart of a method suitable for use in implementing some embodiments of the present disclosure.

FIG. **4** illustrates an example parallel processing unit suitable for use in implementing some embodiments of the present disclosure.

FIG. **5**A is a conceptual diagram of a processing system implemented using the PPU of FIG. **4**, suitable for use in implementing some embodiments of the present disclosure.

FIG. **5**B illustrates an exemplary system in which the various architecture and/or functionality of the various previous embodiments may be implemented.

FIG. **5**C illustrates components of an exemplary system that can be used to train and utilize machine learning, in at least one embodiment.

FIG. **6** illustrates an exemplary streaming system suitable for use in implementing some embodiments of the present disclosure.

DETAILED DESCRIPTION

Systems and methods are disclosed related to lithography simulation using a neural network. As the semiconductor industry continues to shrink integrated circuit geometries, requirements for efficient and high-quality turnaround in manufacturability-aware layout design and optimization have increased. Lithography simulation has developed to ensure that the masks used to fabricate the circuits satisfy the chip yield and fabrication turnaround time targets. To manufacture an integrated circuit (chip), an initial layout for the integrated circuit design is processed to compute a wafer image (e.g., resist material "printed" on the wafer using photomasks). Lithography simulation processes the initial layout according to optical physics to compute an estimated wafer image without actually constructing the physical masks or consuming any wafer fabrication resources. Lithography simulation may be used to confirm manufacturability of the design layout before it is fabricated and is a critical procedure in design for manufacturability (DFM) flows as it significantly impacts the reliability and efficiently of mask optimization and layout printability estimation.

A machine learning model may be used to complete the contour prediction efficiently and accurately. To address the concerns with conventional solutions, the dual-band neural network includes of two perception paths or branches that separately process an input layout (mask image). A first branch extracts low-frequency global information from the mask image and a second branch extracts high-frequency local information from the mask image. In contrast with conventional convolutional neural network-based solutions that lack the ability to capture the necessary global information important to accurately estimate lithography behavior, the dual-band neural network leverages both local high-frequency and global low-frequency components of the mask image for high lithography simulation accuracy. Furthermore, frequency domain representations may more accurately and efficiently represent global layout characteristics. Therefore, for high-quality lithography modeling, a Fourier Neural Operator (FNO) may be used within the first branch of the dual-band neural network.

FIG. **1**A illustrates a block diagram of a dual-band neural network lithography simulation system **100**, in accordance with an embodiment. It should be understood that this and other arrangements described herein are set forth only as examples. Other arrangements and elements (e.g., machines, interfaces, functions, orders, groupings of functions, etc.) may be used in addition to or instead of those shown, and some elements may be omitted altogether. Further, many of the elements described herein are functional entities that may be implemented as discrete or distributed components or in conjunction with other components, and in any suitable combination and location. Various functions described herein as being performed by entities may be carried out by hardware, firmware, and/or software. For instance, various functions may be carried out by a processor executing instructions stored in memory. Furthermore, persons of ordinary skill in the art will understand that any system that performs the operations of the dual-band neural network lithography simulation system **100** is within the scope and spirit of embodiments of the present disclosure.

The dual-band neural network lithography simulation system **100** includes a low-frequency extraction branch **110**, a high-frequency extraction branch **115**, and a reconstruction unit **120**. The dual-band neural network lithography simulation system **100** receives a mask image **105**, such as the mask image **106** and produces an estimated fabrication image **125**, such as resist image **126**. The mask image is an initial layout for an integrated circuit, such as a shape layout for layers of a chip design generated by a computer aided design tool.

Conventional lithography simulation also uses a mask image, but generates an intermediate image, specifically, an aerial image **130** to produce the resist image **125**. The dual-band neural network lithography simulation system **100** does not rely on the aerial image **130** and instead generates an estimated fabrication image **125** directly from the mask image **105**. The conventional aerial image **130** is generated using an optical model that estimates light intensities at the wafer resulting from light intensities (photons/second) projected through the physical mask. The aerial image **130** accounts for interactions with the light as it is projected through the physical mask and characteristics of the projection device (e.g., lenses, light source, etc.).

The low-frequency extraction branch **110** comprises a global perception path to capture mask image **105** layout semantic information for forming resist contours. Low-frequency component features **112** may represent the layout semantic information. In an embodiment, the low-frequency extraction branch **110** comprises a reduced FNO that is compliant with a realistic optical model for mask layout global perception (GP). Although FNOs offer a great advantage for modeling physical equations, FNOs are not directly applicable to lithography simulation tasks due to the large computation cost induced by multiple Fourier Transforms on extremely high-resolution images (2000×2000 or more pixels), such as full chip mask images. Therefore, the FNO is adapted to provide the reduced FNO that reduces the computational cost while still attaining the necessary information.

Although the reduced FNO is designed to capture the semantic information of mask images **105**, the reduced FNO alone is not sufficient for precise lithography modeling. The primary reason is that the reduced FNO drops most high-frequency components of the mask image **105** which contribute to detail formation and are equally important as semantic information. Therefore, the dual-band neural network lithography simulation system **100** also includes the high-frequency extraction branch **115**. In an embodiment, the high-frequency extraction branch **115** comprises a convolutional local perception (LP) path to compensate for the detail loss introduced with the reduced FNO optimization. In an embodiment, the high-frequency extraction branch **115** is a deep convolutional neural network with multiple layers, such as a visual geometry group (VGG) architecture. The

high-frequency extraction branch **115** generates high-frequency component features **118** representing learned feature maps carrying high-frequency mask content.

The reconstruction unit **120** task is to collect and use global and local perception features to reconstruct resist contours. In an embodiment, each of the low-frequency extraction branch **110**, the high-frequency extraction branch **115**, and the reconstruction unit **120** is implemented as a neural network model. In an embodiment, the reconstruction unit **120** comprises a convolution-based image reconstruction (IR) path that concatenates the low-frequency component features **112** and the high-frequency component features **118**, and generates the estimated fabrication image **125**, such as a desired resist image **125**. In an embodiment, the estimated fabrication image **125** is an estimated image of photoresist resulting from fabrication using the input mask image **105**. In an embodiment, layer features **116** generated by separate layers of the high-frequency extraction branch **115** are input to the reconstruction unit **120**. In an embodiment, layer features **116** output by earlier layers are input to later layers of the reconstruction unit **120** and layer features **116** output by later layers are input to earlier layers of the reconstruction unit **120**. In an embodiment, the reconstruction unit **120** comprises a series of transposed convolution layers to rescale low-level feature maps back to the original mask size followed by single-strided convolution layers for contour refinement. The two branches (the low-frequency extraction branch **110** and the high-frequency extraction branch **115**) and the reconstruction unit **120** together form the dual-band neural network lithography simulation system **100**.

The function of the reduced FNO resembles that of the physical lithography model, as described further herein. In the context of the following description, the following basic terminologies and the problem formulation are used. Lowercase letters (e.g. x) represent scalars, bold lowercase letters represent vectors (e.g. x), bold uppercase letters (e.g. X) represent matrices, $\otimes$ represents 2D convolution operations and $\odot$ for element-wise production. $\mathcal{F}$ and $\mathcal{F}^{-1}$ represent Fourier and inverse Fourier transform, respectively.

The Hopkins diffraction model is well accepted in literature to represent lithography behavior and may therefore be used as the physical lithography model. However, computing the Hopkins diffraction model is extremely time consuming. To reduce the compute overhead, a singular value decomposition (SVD) approximation is typically adopted for lithography modeling. The basic idea is to take the SVD of the coefficient matrix in the Hopkins diffraction model and formulate the lithography forward process as

$$I(m, n) = \sum_{k=1}^{N^2} \alpha_k |h_k(m, n) \otimes M(m, n)|^2, \quad \text{eq. (1)}$$

where h_k terms are lithography kernels and α_k terms are the associated eigenvalues. If only the l largest α_k values are kept for faster calculation, equation (1) becomes

$$I(m, n) = \sum_{k=1}^{l} \alpha_k |h_k(m, n) \otimes M(m, n)|^2, \; l \ll N^2. \quad \text{eq. (2)}$$

The computing cost can be further reduced by moving to Fourier space as

$$I = \sum_{k=1}^{l} \alpha_k |\mathcal{F}^{-1}(\mathcal{F}(h_k) \odot \mathcal{F}(M))|^2, \quad \text{eq. (3)}$$

which is normally the equation used for forward simulation. Note that the key components in equation (3) include a series of Fourier transforms, linear operations on Fourier coefficients and inverse Fourier transforms. For simplicity, a constant threshold resist model may be applied to obtain the final wafer contours for the estimated fabrication image **125**.

The operation performed by the dual-band neural network lithography simulation system **100** may be summarized as follows. Given a set of mask images **105** $\mathcal{M}_{tr}=\{M_1, M_2, \ldots, M_n\}$ and their corresponding estimated fabrication (wafer) images **125** $\mathcal{Z}_{rt}^*=\{Z_1^*, Z_2^*, \ldots, Z_n^*\}$, a goal is to design and train a machine learning model $f(\cdot; W)$ such that, for new designs $\mathcal{M}_{te}=\{M_{n+1}, M_{n+2}, \ldots, M_{n+k}\}$ and $\mathcal{Z}_{te}^*=\{Z_{n+1}^*, Z_{n+2}^*, \ldots, Z_{n+k}^*\}$, mIOU $(f(\mathcal{M}_{te}), \mathcal{Z}_{te}^*)$ and mPA$(f(\mathcal{M}_{te}), \mathcal{Z}_{te}^*)$ can be maximized. Mean Intersection Over Union (mIOU) and Mean Pixel Accuracy (mPA) are used to evaluate the performance and may be used to measure semantic segmentation tasks. Lithography modeling may be viewed as a pixel-level classification problem for two classes, namely contour and background.

Given k classes of predicted shapes P_i and their ground truth i=1, 2, . . . k. The mIOU is defined as

$$mIOU(P, G) = \frac{1}{k} \sum_{i=1}^{k} \frac{P_i \cap G_i}{P_i \cup G_i}. \quad \text{eq. (4)}$$

Given k classes of predicted shapes P_i and their ground truth i=1, 2, . . . k. The mPA is defined as

$$mPA(P, G) = \frac{1}{k} \sum_{i=1}^{k} \frac{P_i \cap G_i}{G_i.} \quad \text{eq. (5)}$$

Because the low-frequency extraction branch **110** operates in a mechanism that resembles the physical lithography model as in eq. (3), the dual-band neural network lithography simulation system **100** may be considered optically compliant. In summary, the dual-band neural network lithography simulation system **100** has the following advantages: (1) The low-frequency extraction branch **110** performs an efficient global perception on input mask images **105** from frequency domain analysis. (2) Convolutions in the high-frequency extraction branch **115** compensate for the high-frequency information loss in the low-frequency extraction branch **110** to produce high quality resist contour reconstructions, estimated fabrication images **125**. (3) The optical compliant property of the dual-band neural network lithography simulation system **100** makes it possible to create highly accurate lithography modeling results with a 10× smaller model size compared with conventional solutions.

More illustrative information will now be set forth regarding various optional architectures and features with which the foregoing framework may be implemented, per the desires of the user. It should be strongly noted that the following information is set forth for illustrative purposes and should not be construed as limiting in any manner. Any of the following features may be optionally incorporated with or without the exclusion of other features described.

FIG. **1**B illustrates a flowchart of a method **150** suitable for use in implementing some embodiments of the present disclosure. Each block of method **150**, described herein, comprises a computing process that may be performed using any combination of hardware, firmware, and/or software. For instance, various functions may be carried out by a

processor executing instructions stored in memory. The method may also be embodied as computer-usable instructions stored on computer storage media. The method may be provided by a standalone application, a service or hosted service (standalone or in combination with another hosted service), or a plug-in to another product, to name a few. In addition, method **150** is described, by way of example, with respect to the dual-band neural network lithography simulation system **100** of FIG. **1**A. However, this method may additionally or alternatively be executed by any one system, or any combination of systems, including, but not limited to, those described herein. Furthermore, persons of ordinary skill in the art will understand that any system that performs method **150** is within the scope and spirit of embodiments of the present disclosure.

At step **155**, a mask image defining shapes for fabrication of an integrated circuit is processed by a first branch of a neural network, according to an optical model for lithography, to extract low-frequency component features. In an embodiment, the low-frequency component features correspond to light intensity. In an embodiment, the mask image is downsampled for processing by the first branch. In an embodiment, the downsampled mask image is subdivided into tiles for processing by the first branch. In an embodiment, each one of the tiles at least partially overlaps with an adjacent tile. In an embodiment, the first branch extracts high-frequency component features for each processed tile and further comprising concatenating the high-frequency component features for the processed tiles for input to the reconstruction portion of the neural network.

In an embodiment, the first branch processes the mask image by performing a Fourier transform on the mask image to convert the mask image into a frequency domain, convolving the converted mask image with a channel-lifting operator, performing a linear operation on the convolved converted mask image to extract low-frequency features in the frequency domain, and performing an inverse Fourier transform on the extracted low-frequency features in the frequency domain to produce the low-frequency component features.

At step **160**, high-frequency component features are extracted from the mask image by a second branch of the neural network. In an embodiment, the high-frequency component features correspond to contour and shape edge details. At step **165**, an estimated fabrication image is produced by processing the low-frequency component features and the high-frequency component features by a reconstruction portion of the neural network.

FIG. **2**A illustrates a block diagram of a low-frequency extraction branch **200** of the dual-band neural network lithography simulation system **100** shown in FIG. **1**A suitable for use in implementing some embodiments of the present disclosure. The low-frequency extraction branch **200** may be used in place of the low-frequency extraction branch **110** includes a P unit **201**, multiple Fourier units **205**, and a Q unit **206**.

In an embodiment, the low-frequency extraction branch **200** is a baseline FNO that tries to learn a parameterized mapping $\mathcal{G}$ between two finite dimension spaces from a set of observations such that $\mathcal{G}$ is close to the physical behavior.

$$\mathcal{G}:\mathcal{A}_{\times\Theta}\rightarrow\mathcal{U}, \quad \text{eq. (6)}$$

where Θ is the parameter space. Each mask image **105** and low-frequency component features **112** correspond to a(x) and u(x), respectively, that are observed pairs from $\mathcal{A}$ and $\mathcal{U}$. P is an operation performed by the P unit **201** that lifts a(x) to a higher dimension and a Q operation performed by the Q unit **206** projects the data back to the target dimension.

FIG. **2**B illustrates a block diagram of a Fourier unit **205** suitable for use in implementing some embodiments of the present disclosure. Each Fourier unit **205** includes a Fourier transform unit **210**, a convolution operator defined in the Fourier space along with a bounded linear operation unit **215**, and an inverse Fourier transform unit **220**.

$$v_{t+1}(x)=\sigma((\mathcal{L}\ v_t+\mathcal{F}^{-1}(\mathcal{R}\cdot(\mathcal{F}\ v_t)))(x)), \quad \text{eq. (7)}$$

where $\mathcal{L}$ and $\mathcal{R}$ are channel-wise linear transformations and σ is some element-wise activation function. The channel-wise linear transformation $\mathcal{L}$ is implemented using a bypass link $\mathcal{L}$, as shown in FIG. **2**B. The linear operation unit **215** performs the channel-wise linear transformation $\mathcal{R}$. When applied to image-to-image mapping in a lithography modeling problem, equation (7) will work in a discrete manner. Let the input of each Fourier Unit **205** to be a three-dimensional tensor $V_t\in\mathbb{R}^{C\times H\times W}$, then equation (7) becomes

$$V_{t+1}=\sigma(\boldsymbol{V}_{t,\mathcal{L}}+\boldsymbol{V}_{t,\mathcal{F}}), \quad \text{eq. (8)}$$

where

$$\boldsymbol{V}_{t,\mathcal{L}}=V_t\otimes\boldsymbol{W}_{t,\mathcal{L}},\boldsymbol{W}_{t,\mathcal{L}}\in\mathbb{R}^{1\times C\times 1\times 1}, \quad \text{eq. (9)}$$

$$\boldsymbol{V}_{t,\mathcal{F}}=\mathcal{F}^{-1}(\mathcal{F}(V_t)_{k\text{-}truncated},\boldsymbol{W}_{\mathcal{R}}),\boldsymbol{W}_{\mathcal{R}}\in\mathbb{C}^{C\times C\times H\times W}. \quad \text{eq. (10)}$$

Note that all the Fourier and inverse Fourier transforms are performed on the last two dimensions of each tensor only and $\mathcal{F}(V_t)_{k\text{-}truncated}$ is obtained by only keeping the k lowest frequency components of $\mathcal{F}(V_t)$ and discarding the remaining entries (by setting to zero).

Based on the flow, there are two major concerns for the baseline FNO being an efficient lithography simulator. A first concern is that the "stacked" (sequence of) Fourier units **205** are not compliant with the real physical model that applies a single Fourier transform to a mask image and inverse Fourier transforms to determine light intensity. A second concern is that multiple Fourier and inverse Fourier transforms pose large computation overheads when processing large mask images **105**.

FIG. **2**C illustrates another block diagram of a low-frequency extraction branch of the system shown in FIG. **1**A suitable for use in implementing some embodiments of the present disclosure. To address the concerns with the baseline FNO architecture, a reduced FNO architecture with a single optimized Fourier Unit **250** may be implemented for the low-frequency extraction branch **110**. In an embodiment, the mask image **105** is downsampled by a downsample unit **240** before being processed by the optimized Fourier unit **250**. Downsampling the mask image **105** reduces the amount of image data to be processed and improves efficiency of the low-frequency extraction branch **110**. Furthermore, the optimized Fourier unit **250** discards most high-frequency component, so it is not necessary to process a high-resolution mask image **105**. For large images, the downsampled mask image **105** may be subdivided into multiple overlapping tiles by a tiling unit **245** and each tile is then processed by the optimized Fourier unit **250**. The processed tiles are then recombined by the tile combine unit **255** to produce the low-frequency component features **112** corresponding to the mask image **105**.

FIG. **2**D illustrates a block diagram of the optimized Fourier unit **250** suitable for use in implementing some embodiments of the present disclosure. The optimized Fourier unit **250** includes a Fourier transform unit **260**, a

convolution unit **265**, a linear operation unit **270**, and an inverse Fourier transform unit **275**. When the mask image **105** is subdivided into tiles, the optimized Fourier unit **250** processes each tile mask image **208** to produce tile low-frequency component features **212** for the tile mask image **208**. Because Fourier transforms are applied channel-wise and preserve linearity, the optimized Fourier Unit **250** performs Fourier transforms prior to the channel-lifting operation P. Thus, the computation performed by the optimized Fourier unit **250** becomes

$$V=\sigma(\mathcal{F}^{-1}(\mathcal{F}(A)_{k\text{-}truncated}\otimes W_P.\boldsymbol{W}_{\mathcal{R}})). \quad \text{eq. (11)}$$

where $A\in\mathbb{R}^{1\times H\times W}$ is the input image, and $W_P\in\mathbb{C}^{1\times C\times 1\times 1}$ is the convolution kernel used by the convolution unit **265** to perform channel-lifting. The linear operation unit **270** performs the channel-wise linear transformation $\mathcal{R}$. Because the low-frequency extraction branch **110** shown in FIG. **2**C includes only one optimized Fourier unit **250**, the bypass link $\mathcal{L}$ becomes less necessary and the bypass link may be discarded to simplify the low-frequency extraction branch **110**. With these optimizations, equation (11) attains the same expressive power as equation (10) while saving a significant amount of calculation. In equations (10) and (11), convolution operations cost $\mathcal{O}(C^2HW)$ because typically C<<H,W. Assuming both are implemented with Fast Fourier Transforms (FFT) and complete in $\mathcal{O}$ (HWlog(HW)), FFTs would dominate the computation. Given that FFTs are performed channel-wise, the optimized Fourier unit **250** defined in equation (11) can save ~50% of runtime by avoiding C FFT operations.

In an embodiment, the low-frequency extraction branch **110** includes the optimized Fourier unit **250** that significantly reduces computation compared to the Fourier unit **205** in the baseline FNO, low-frequency extraction branch **200**, while extracting necessary global information to analyze lithography behavior. The optimized Fourier unit **250** resembles physical lithography equations, ensuring faster training convergence and contour generation quality. In contrast with the low-frequency extraction branch **110**, the high-frequency extraction branch **115** uses convolutional layers to capture mask image details and compensate for information loss in the low-frequency extraction branch **110**. In an embodiment, the dual-band neural network lithography simulation system **100** is an optically compliant neural network designed for high resolution metal/via layer lithography modeling at 1 nm^2/pixel.

Because the dual-band neural network lithography simulation system **100** is differentiable, it can be trained using supervised learning and evaluate with fix-sized layout clips (sub-region of a full size chip layout) that are not equivalent to full size chip layouts. However, the dual-band neural network lithography simulation system **100** may also perform a more challenging and practical task: full chip simulation. The high-frequency extraction branch **115** may process a mask image **105** of any dimensions to extract the high-frequency features or contour and shape edge details. In an embodiment, the mask image **105** is downsampled for processing by the low-frequency extraction branch **110**. In an embodiment, the (downsampled) mask image **105** is subdivided into overlapping tiles for processing by the low-frequency extraction branch **110**.

Although the dual-band neural network lithography simulation system **100** allows for any-sized mask image **105**, the generated estimated fabrication image **125** quality may be seriously affected if a size of the mask image **105** is much larger than the tile size used for training. This is because the weights in the Fourier units **205** or the optimized Fourier unit **250** are trained for the k-lowest frequency components of a smaller tile and will result in significant information loss if applied on scaled inputs.

FIG. **3**A illustrates a conceptual diagram of a tiling a full chip core **304** for processing by the low-frequency extraction branch **110**, in accordance with an embodiment. A mask image **105** for a full chip **303** may be defined as a full chip core **304** including the entire full chip mask pattern and a boundary region surrounding the full chip core **304** that does not include any mask pattern.

Intuitively, clips or mask images **105** of full chip layouts can be subdivided into tiles that have the same size as the clips used for training the dual-band neural network lithography simulation system **100**. This is, however, not as simple as it looks due to the concept of optical diameter. Informed by lithography physics, the light intensity on the resist stage can be viewed as superposition of multiple orders of diffraction patterns. As a result, the light intensity at a certain wafer location is determined by a large area of surrounding mask patterns. The size of the area is usually measured by its diameter, i.e., the optical diameter. For this reason, simply cutting the full chip core **304** into clips is not feasible as we must account for shapes near the boundaries of the clips. Therefore, the chip **303** is subdivided or partitioned tiles **301** that at least partially overlap. The region at the center of each tile **301** is a tile core **302** or mask region for simulation and shapes outside the tile core **302** are ignored as they are too close to the boundaries of the tile **301**. The basic idea is to subdivide the full chip **303** into small tiles which have the same size as is used for training, input to the Fourier units **205** or the optimized Fourier unit **250** in batches. In an embodiment, the tiles **301** are half overlapped with each other such that the tile cores **302** in the tiles **301** will exactly cover the full chip core **304** of the full chip **303**.

When the mask image **105** is downsampled by the low-frequency extraction branch **110**, the downsampled mask image generated by the downsample unit **240** is subdivided into the tiles **301** by the tiling unit **245**. In an embodiment, the mask image **105** is downsampled by eight in each dimension. After processing by the optimized Fourier unit **250**, the low-frequency component features for each processed tile may be concatenated back to the full chip size by the tile combine unit **255**. The convolution and transposed convolution layers in the reconstruction unit **120** then process the low-frequency component features **112** and the high-frequency component features **118** for the full chip.

The full chip global perception can be mathematically described in the following manner. Suppose the dual-band neural network lithography simulation system **100** is trained with H×W mask-contour pairs and the optical diameter is d. The global perception

$$\mathcal{G}: \mathbb{R}^{H\times W}\rightarrow\mathbb{R}^{C\times\frac{H}{8}\times\frac{W}{8}}$$

can be expressed as

$$F_{gp}=\mathcal{G}(M;\mathbf{W}_{\mathcal{P}},\mathbf{W}_{\mathcal{R}}) \quad \text{eq. (12)}$$

where F_{gp} E

$$\mathbb{R}^{C\times\frac{H}{8}\times\frac{W}{8}}$$

is the feature map output of the low-frequency extraction branch **110**, $M\in\mathbb{R}^{H\times W}$ denotes the downsampled mask

image **105** and $\mathbf{W}_{\mathcal{P}}$, $\mathbf{W}_{\mathcal{R}}$ are trained parameters of the optimized Fourier unit **250**. Let

$$\mathcal{G}_s: \mathbb{R}^{sH\times sW} \to \mathbb{R}^{C\times\frac{sH}{8}\times\frac{sW}{8}}$$

be the low-frequency extraction branch **110** which processes a mask image **105** $M_s \in \mathbb{R}^{sH\times sW}$ that is s× larger than tiles used for training. Then each entry of output feature map $F_{s,g,p}$ is defined as

$$F_{s,g,p}[:, i, j] = \mathcal{G}_s(M_s; W_{\mathcal{P}}, W_{\mathcal{R}})[:, i, j] \quad \text{eq. (13)}$$

$$= \mathcal{G}\left(M_s\left[\frac{mH}{2}:\frac{(m+2)H}{2}, \frac{nW}{2}:\frac{(n+2)W}{2}\right]; W_{\mathcal{P}}, W_{\mathcal{R}}\right)[:, p, q]$$

$$\text{where } \frac{d}{2} \le i < sH - \frac{d}{2}, \frac{d}{2} \le j < sW - \frac{d}{2} \text{ and} \quad \text{eq. (14)}$$

$$m = \left\lfloor \frac{2\left(i-\frac{d}{2}\right)}{H} \right\rfloor, n = \left\lfloor \frac{2\left(j-\frac{d}{2}\right)}{W} \right\rfloor,$$

$$p = \left(\left(i-\frac{d}{2}\right) \bmod \frac{H}{2}\right) + \frac{d}{2},$$

$$q = \left(\left(j-\frac{d}{2}\right) \bmod \frac{W}{2}\right) + \frac{d}{2}.$$

It should be noted that such a full chip lithography simulation scheme enables training of the dual-band neural network lithography simulation system **100** with small sized tiles **301** and evaluated to produce an estimated fabrication image **125** on any sized mask image **105** without performance degradation. In an embodiment, the tile size of a tile **301** matches the tile size used during training of the dual-band neural network lithography simulation system **100**.

The dual-band neural network lithography simulation system **100** can efficiently leverage global and local mask image **105** characteristics for estimated fabrication image **125** construction at the resolution of 1 nm^2/pixel with any tile size. The optimized Fourier unit **250** in the low-frequency extraction branch **110** can accommodate large-scale layout mask image **105** inputs at reduced computational costs, resulting in a 20× smaller neural network lithography model size and 10× faster training convergence compared to a conventional neural network-based lithography model. Furthermore, the dual-band neural network lithography simulation system **100** is adaptable to both metal and via layer designs with significant better contour prediction quality compared to conventional machine learning-based lithography models.

FIG. **3**B illustrates a flowchart of a method for performing step **155** of the method **150** shown in FIG. **1**B, in accordance with an embodiment. Step **155** may be performed by the low-frequency extraction branch **200** or the low-frequency extraction branch **110**. At step **320**, a Fourier transform is performed on the mask image **105** to convert the mask image **105** into a frequency domain. In an embodiment, the mask image **105** may be downsampled before the Fourier transform is performed. In an embodiment, the (downsampled or original) mask image **105** may be tiled and the step **155** may be performed for each tile of the mask image **105**.

At step **325**, the converted mask image is convolved with a channel-lifting operator. At step **330**, a linear operation is performed on the convolved converted mask image to extract low-frequency features in the frequency domain. At step **335**, an inverse Fourier transform is performed on the extracted low-frequency features in the frequency domain to produce the low-frequency component features **112**. In an embodiment, when the mask image **105** is tiled, the low-frequency component features **112** comprises tile low-frequency component features **212** for each tile. In an embodiment, each channel of the low-frequency component features **212** captures an intensity map that closely matches a corresponding aerial image. The high-frequency component features **118** comprises feature maps representing shape edges and contour details.

In an embodiment, the dual-band neural network lithography simulation system **100** outperforms conventional machine learning models in terms of mPA and mIOU on both metal and via layers and on both low and high resolution mask images. For the via cases, the dual-band neural network lithography simulation system **100** provides slightly better results because via designs are regular and relatively easy to learn. However, for metal shapes, results show a larger performance gap between the dual-band neural network lithography simulation system **100** and conventional machine learning models. For even smaller integrated circuit geometries, such as 14 nm and below, the dual-band neural network lithography simulation system **100** exhibits much more advantages in terms of shorter training time and accuracy over conventional solutions.

FIG. **3**C illustrates a flowchart of a method suitable for use in implementing some embodiments of the present disclosure. In an embodiment, the method **350** performs a reverse flow compared with the method **150** shown in FIG. **1**B. Each block of method **350**, described herein, comprises a computing process that may be performed using any combination of hardware, firmware, and/or software. For instance, various functions may be carried out by a processor executing instructions stored in memory. The method may also be embodied as computer-usable instructions stored on computer storage media. The method may be provided by a standalone application, a service or hosted service (standalone or in combination with another hosted service), or a plug-in to another product, to name a few. In addition, method **350** is described, by way of example, with respect to the dual-band neural network lithography simulation system **100** of FIG. **1**A. However, this method may additionally or alternatively be executed by any one system, or any combination of systems, including, but not limited to, those described herein. Furthermore, persons of ordinary skill in the art will understand that any system that performs method **350** is within the scope and spirit of embodiments of the present disclosure.

At step **355**, a fabrication image defining patterns for an integrated circuit is processed by a first branch of a neural network according to an optical model for lithography to extract low-frequency component features. In an embodiment, the fabrication image is an image of a fabricated integrated circuit wafer. At step **360**, high-frequency component features are extracted from the fabrication image by a second branch of the neural network.

At step **365**, an estimated mask image is produced by processing the low-frequency component features and the high-frequency component features by a reconstruction portion of the neural network. In an embodiment, the estimated mask image is an estimate of the mask image used to manufacture the integrated circuit implemented by the fabrication image.

The dual-band neural network lithography simulation system **100** receives an input mask image and outputs an estimated fabrication image. Separate branches of the dual-band neural network lithography simulation system **100**

extract low-frequency features or light intensity information from the input image and high-frequency features representing contour and shape edge details. The high-frequency extraction branch includes an optimized Fourier transform unit rather than purely convolutional processing layers. Both the low-frequency and the high-frequency features contribute to the estimated fabrication image. In a reverse operation embodiment, the dual-band neural network lithography simulation system **100** receives an input fabrication image and outputs an estimated mask image. In addition to providing a solution for performing lithography simulation of full-chip designs, the dual-band neural network lithography simulation system **100** produces accurate results more efficiently compared with conventional solutions.

Parallel Processing Architecture

FIG. **4** illustrates a parallel processing unit (PPU) **400**, in accordance with an embodiment. In an embodiment, a processor such as the PPU **400** may be configured to implement a neural network model, such as the dual-band neural network lithography simulation system **100**. In an embodiment, the PPU **400** implements one or more of the low-frequency extraction branch **110**, the high-frequency extraction branch **115**, and the reconstruction unit **120**. The neural network model may be implemented as software instructions executed by the processor or, in other embodiments, the processor can include a matrix of hardware elements configured to process a set of inputs (e.g., electrical signals representing values) to generate a set of outputs, which can represent activations of the neural network model. In yet other embodiments, the neural network model can be implemented as a combination of software instructions and processing performed by a matrix of hardware elements. Implementing the neural network model can include determining a set of parameters for the neural network model through, e.g., supervised or unsupervised training of the neural network model as well as, or in the alternative, performing inference using the set of parameters to process novel sets of inputs.

In an embodiment, the PPU **400** is a multi-threaded processor that is implemented on one or more integrated circuit devices. The PPU **400** is a latency hiding architecture designed to process many threads in parallel. A thread (e.g., a thread of execution) is an instantiation of a set of instructions configured to be executed by the PPU **400**. In an embodiment, the PPU **400** is a graphics processing unit (GPU) configured to implement a graphics rendering pipeline for processing three-dimensional (3D) graphics data in order to generate two-dimensional (2D) image data for display on a display device. In other embodiments, the PPU **400** may be utilized for performing general-purpose computations. While one exemplary parallel processor is provided herein for illustrative purposes, it should be strongly noted that such processor is set forth for illustrative purposes only, and that any processor may be employed to supplement and/or substitute for the same.

One or more PPUs **400** may be configured to accelerate thousands of High Performance Computing (HPC), data center, cloud computing, and machine learning applications. The PPU **400** may be configured to accelerate numerous deep learning systems and applications for autonomous vehicles, simulation, computational graphics such as ray or path tracing, deep learning, high-accuracy speech, image, and text recognition systems, intelligent video analytics, molecular simulations, drug discovery, disease diagnosis, weather forecasting, big data analytics, astronomy, molecular dynamics simulation, financial modeling, robotics, factory automation, real-time language translation, online search optimizations, and personalized user recommendations, and the like.

As shown in FIG. **4**, the PPU **400** includes an Input/Output (I/O) unit **405**, a front end unit **415**, a scheduler unit **420**, a work distribution unit **425**, a hub **430**, a crossbar (Xbar) **470**, one or more general processing clusters (GPCs) **450**, and one or more memory partition units **480**. The PPU **400** may be connected to a host processor or other PPUs **400** via one or more high-speed NVLink **410** interconnect. The PPU **400** may be connected to a host processor or other peripheral devices via an interconnect **402**. The PPU **400** may also be connected to a local memory **404** comprising a number of memory devices. In an embodiment, the local memory may comprise a number of dynamic random access memory (DRAM) devices. The DRAM devices may be configured as a high-bandwidth memory (HBM) subsystem, with multiple DRAM dies stacked within each device.

The NVLink **410** interconnect enables systems to scale and include one or more PPUs **400** combined with one or more CPUs, supports cache coherence between the PPUs **400** and CPUs, and CPU mastering. Data and/or commands may be transmitted by the NVLink **410** through the hub **430** to/from other units of the PPU **400** such as one or more copy engines, a video encoder, a video decoder, a power management unit, etc. (not explicitly shown). The NVLink **410** is described in more detail in conjunction with FIG. **5**B.

The I/O unit **405** is configured to transmit and receive communications (e.g., commands, data, etc.) from a host processor (not shown) over the interconnect **402**. The I/O unit **405** may communicate with the host processor directly via the interconnect **402** or through one or more intermediate devices such as a memory bridge. In an embodiment, the I/O unit **405** may communicate with one or more other processors, such as one or more the PPUs **400** via the interconnect **402**. In an embodiment, the I/O unit **405** implements a Peripheral Component Interconnect Express (PCIe) interface for communications over a PCIe bus and the interconnect **402** is a PCIe bus. In alternative embodiments, the I/O unit **405** may implement other types of well-known interfaces for communicating with external devices.

The I/O unit **405** decodes packets received via the interconnect **402**. In an embodiment, the packets represent commands configured to cause the PPU **400** to perform various operations. The I/O unit **405** transmits the decoded commands to various other units of the PPU **400** as the commands may specify. For example, some commands may be transmitted to the front end unit **415**. Other commands may be transmitted to the hub **430** or other units of the PPU **400** such as one or more copy engines, a video encoder, a video decoder, a power management unit, etc. (not explicitly shown). In other words, the I/O unit **405** is configured to route communications between and among the various logical units of the PPU **400**.

In an embodiment, a program executed by the host processor encodes a command stream in a buffer that provides workloads to the PPU **400** for processing. A workload may comprise several instructions and data to be processed by those instructions. The buffer is a region in a memory that is accessible (e.g., read/write) by both the host processor and the PPU **400**. For example, the I/O unit **405** may be configured to access the buffer in a system memory connected to the interconnect **402** via memory requests transmitted over the interconnect **402**. In an embodiment, the host processor writes the command stream to the buffer and then transmits a pointer to the start of the command stream to the PPU **400**. The front end unit **415** receives pointers to one or

more command streams. The front end unit **415** manages the one or more streams, reading commands from the streams and forwarding commands to the various units of the PPU **400**.

The front end unit **415** is coupled to a scheduler unit **420** that configures the various GPCs **450** to process tasks defined by the one or more streams. The scheduler unit **420** is configured to track state information related to the various tasks managed by the scheduler unit **420**. The state may indicate which GPC **450** a task is assigned to, whether the task is active or inactive, a priority level associated with the task, and so forth. The scheduler unit **420** manages the execution of a plurality of tasks on the one or more GPCs **450**.

The scheduler unit **420** is coupled to a work distribution unit **425** that is configured to dispatch tasks for execution on the GPCs **450**. The work distribution unit **425** may track a number of scheduled tasks received from the scheduler unit **420**. In an embodiment, the work distribution unit **425** manages a pending task pool and an active task pool for each of the GPCs **450**. As a GPC **450** finishes the execution of a task, that task is evicted from the active task pool for the GPC **450** and one of the other tasks from the pending task pool is selected and scheduled for execution on the GPC **450**. If an active task has been idle on the GPC **450**, such as while waiting for a data dependency to be resolved, then the active task may be evicted from the GPC **450** and returned to the pending task pool while another task in the pending task pool is selected and scheduled for execution on the GPC **450**.

In an embodiment, a host processor executes a driver kernel that implements an application programming interface (API) that enables one or more applications executing on the host processor to schedule operations for execution on the PPU **400**. In an embodiment, multiple compute applications are simultaneously executed by the PPU **400** and the PPU **400** provides isolation, quality of service (QoS), and independent address spaces for the multiple compute applications. An application may generate instructions (e.g., API calls) that cause the driver kernel to generate one or more tasks for execution by the PPU **400**. The driver kernel outputs tasks to one or more streams being processed by the PPU **400**. Each task may comprise one or more groups of related threads, referred to herein as a warp. In an embodiment, a warp comprises 32 related threads that may be executed in parallel. Cooperating threads may refer to a plurality of threads including instructions to perform the task and that may exchange data through shared memory. The tasks may be allocated to one or more processing units within a GPC **450** and instructions are scheduled for execution by at least one warp.

The work distribution unit **425** communicates with the one or more GPCs **450** via XBar **470**. The XBar **470** is an interconnect network that couples many of the units of the PPU **400** to other units of the PPU **400**. For example, the XBar **470** may be configured to couple the work distribution unit **425** to a particular GPC **450**. Although not shown explicitly, one or more other units of the PPU **400** may also be connected to the XBar **470** via the hub **430**.

The tasks are managed by the scheduler unit **420** and dispatched to a GPC **450** by the work distribution unit **425**. The GPC **450** is configured to process the task and generate results. The results may be consumed by other tasks within the GPC **450**, routed to a different GPC **450** via the XBar **470**, or stored in the memory **404**. The results can be written to the memory **404** via the memory partition units **480**, which implement a memory interface for reading and writing data to/from the memory **404**. The results can be transmitted to another PPU **400** or CPU via the NVLink **410**. In an embodiment, the PPU **400** includes a number U of memory partition units **480** that is equal to the number of separate and distinct memory devices of the memory **404** coupled to the PPU **400**. Each GPC **450** may include a memory management unit to provide translation of virtual addresses into physical addresses, memory protection, and arbitration of memory requests. In an embodiment, the memory management unit provides one or more translation lookaside buffers (TLBs) for performing translation of virtual addresses into physical addresses in the memory **404**.

In an embodiment, the memory partition unit **480** includes a Raster Operations (ROP) unit, a level two (L2) cache, and a memory interface that is coupled to the memory **404**. The memory interface may implement 32, 64, 128, 1024-bit data buses, or the like, for high-speed data transfer. The PPU **400** may be connected to up to Y memory devices, such as high bandwidth memory stacks or graphics double-data-rate, version 5, synchronous dynamic random access memory, or other types of persistent storage. In an embodiment, the memory interface implements an HBM2 memory interface and Y equals half U. In an embodiment, the HBM2 memory stacks are located on the same physical package as the PPU **400**, providing substantial power and area savings compared with conventional GDDR5 SDRAM systems. In an embodiment, each HBM2 stack includes four memory dies and Y equals 4, with each HBM2 stack including two 128-bit channels per die for a total of 8 channels and a data bus width of 1024 bits.

In an embodiment, the memory **404** supports Single-Error Correcting Double-Error Detecting (SECDED) Error Correction Code (ECC) to protect data. ECC provides higher reliability for compute applications that are sensitive to data corruption. Reliability is especially important in large-scale cluster computing environments where PPUs **400** process very large datasets and/or run applications for extended periods.

In an embodiment, the PPU **400** implements a multi-level memory hierarchy. In an embodiment, the memory partition unit **480** supports a unified memory to provide a single unified virtual address space for CPU and PPU **400** memory, enabling data sharing between virtual memory systems. In an embodiment the frequency of accesses by a PPU **400** to memory located on other processors is traced to ensure that memory pages are moved to the physical memory of the PPU **400** that is accessing the pages more frequently. In an embodiment, the NVLink **410** supports address translation services allowing the PPU **400** to directly access a CPU's page tables and providing full access to CPU memory by the PPU **400**.

In an embodiment, copy engines transfer data between multiple PPUs **400** or between PPUs **400** and CPUs. The copy engines can generate page faults for addresses that are not mapped into the page tables. The memory partition unit **480** can then service the page faults, mapping the addresses into the page table, after which the copy engine can perform the transfer. In a conventional system, memory is pinned (e.g., non-pageable) for multiple copy engine operations between multiple processors, substantially reducing the available memory. With hardware page faulting, addresses can be passed to the copy engines without worrying if the memory pages are resident, and the copy process is transparent.

Data from the memory **404** or other system memory may be fetched by the memory partition unit **480** and stored in the L2 cache **460**, which is located on-chip and is shared

between the various GPCs **450**. As shown, each memory partition unit **480** includes a portion of the L2 cache associated with a corresponding memory **404**. Lower level caches may then be implemented in various units within the GPCs **450**. For example, each of the processing units within a GPC **450** may implement a level one (L1) cache. The L1 cache is private memory that is dedicated to a particular processing unit. The L2 cache **460** is coupled to the memory interface **470** and the XBar **470** and data from the L2 cache may be fetched and stored in each of the L1 caches for processing.

In an embodiment, the processing units within each GPC **450** implement a SIMD (Single-Instruction, Multiple-Data) architecture where each thread in a group of threads (e.g., a warp) is configured to process a different set of data based on the same set of instructions. All threads in the group of threads execute the same instructions. In another embodiment, the processing unit implements a SIMT (Single-Instruction, Multiple Thread) architecture where each thread in a group of threads is configured to process a different set of data based on the same set of instructions, but where individual threads in the group of threads are allowed to diverge during execution. In an embodiment, a program counter, call stack, and execution state is maintained for each warp, enabling concurrency between warps and serial execution within warps when threads within the warp diverge. In another embodiment, a program counter, call stack, and execution state is maintained for each individual thread, enabling equal concurrency between all threads, within and between warps. When execution state is maintained for each individual thread, threads executing the same instructions may be converged and executed in parallel for maximum efficiency.

Cooperative Groups is a programming model for organizing groups of communicating threads that allows developers to express the granularity at which threads are communicating, enabling the expression of richer, more efficient parallel decompositions. Cooperative launch APIs support synchronization amongst thread blocks for the execution of parallel algorithms. Conventional programming models provide a single, simple construct for synchronizing cooperating threads: a barrier across all threads of a thread block (e.g., the syncthreads( ) function). However, programmers would often like to define groups of threads at smaller than thread block granularities and synchronize within the defined groups to enable greater performance, design flexibility, and software reuse in the form of collective group-wide function interfaces.

Cooperative Groups enables programmers to define groups of threads explicitly at sub-block (e.g., as small as a single thread) and multi-block granularities, and to perform collective operations such as synchronization on the threads in a cooperative group. The programming model supports clean composition across software boundaries, so that libraries and utility functions can synchronize safely within their local context without having to make assumptions about convergence. Cooperative Groups primitives enable new patterns of cooperative parallelism, including producer-consumer parallelism, opportunistic parallelism, and global synchronization across an entire grid of thread blocks.

Each processing unit includes a large number (e.g., 128, etc.) of distinct processing cores (e.g., functional units) that may be fully-pipelined, single-precision, double-precision, and/or mixed precision and include a floating point arithmetic logic unit and an integer arithmetic logic unit. In an embodiment, the floating point arithmetic logic units implement the IEEE 754-2008 standard for floating point arithmetic. In an embodiment, the cores include 64 single-precision (32-bit) floating point cores, 64 integer cores, 32 double-precision (64-bit) floating point cores, and 8 tensor cores.

Tensor cores configured to perform matrix operations. In particular, the tensor cores are configured to perform deep learning matrix arithmetic, such as GEMM (matrix-matrix multiplication) for convolution operations during neural network training and inferencing. In an embodiment, each tensor core operates on a 4×4 matrix and performs a matrix multiply and accumulate operation D=A×B+C, where A, B, C, and D are 4×4 matrices.

In an embodiment, the matrix multiply inputs A and B may be integer, fixed-point, or floating point matrices, while the accumulation matrices C and D may be integer, fixed-point, or floating point matrices of equal or higher bitwidths. In an embodiment, tensor cores operate on one, four, or eight bit integer input data with 32-bit integer accumulation. The 8-bit integer matrix multiply requires 1024 operations and results in a full precision product that is then accumulated using 32-bit integer addition with the other intermediate products for a 8×8×16 matrix multiply. In an embodiment, tensor Cores operate on 16-bit floating point input data with 32-bit floating point accumulation. The 16-bit floating point multiply requires 64 operations and results in a full precision product that is then accumulated using 32-bit floating point addition with the other intermediate products for a 4×4×4 matrix multiply. In practice, Tensor Cores are used to perform much larger two-dimensional or higher dimensional matrix operations, built up from these smaller elements. An API, such as CUDA 9 C++ API, exposes specialized matrix load, matrix multiply and accumulate, and matrix store operations to efficiently use Tensor Cores from a CUDA-C++ program. At the CUDA level, the warp-level interface assumes 16×16 size matrices spanning all 32 threads of the warp.

Each processing unit may also comprise M special function units (SFUs) that perform special functions (e.g., attribute evaluation, reciprocal square root, and the like). In an embodiment, the SFUs may include a tree traversal unit configured to traverse a hierarchical tree data structure. In an embodiment, the SFUs may include texture unit configured to perform texture map filtering operations. In an embodiment, the texture units are configured to load texture maps (e.g., a 2D array of texels) from the memory **404** and sample the texture maps to produce sampled texture values for use in shader programs executed by the processing unit. In an embodiment, the texture maps are stored in shared memory that may comprise or include an L1 cache. The texture units implement texture operations such as filtering operations using mip-maps (e.g., texture maps of varying levels of detail). In an embodiment, each processing unit includes two texture units.

Each processing unit also comprises N load store units (LSUs) that implement load and store operations between the shared memory and the register file. Each processing unit includes an interconnect network that connects each of the cores to the register file and the LSU to the register file, shared memory. In an embodiment, the interconnect network is a crossbar that can be configured to connect any of the cores to any of the registers in the register file and connect the LSUs to the register file and memory locations in shared memory.

The shared memory is an array of on-chip memory that allows for data storage and communication between the processing units and between threads within a processing unit. In an embodiment, the shared memory comprises 128

KB of storage capacity and is in the path from each of the processing units to the memory partition unit **480**. The shared memory can be used to cache reads and writes. One or more of the shared memory, L1 cache, L2 cache, and memory **404** are backing stores.

Combining data cache and shared memory functionality into a single memory block provides the best overall performance for both types of memory accesses. The capacity is usable as a cache by programs that do not use shared memory. For example, if shared memory is configured to use half of the capacity, texture and load/store operations can use the remaining capacity. Integration within the shared memory enables the shared memory to function as a high-throughput conduit for streaming data while simultaneously providing high-bandwidth and low-latency access to frequently reused data.

When configured for general purpose parallel computation, a simpler configuration can be used compared with graphics processing. Specifically, fixed function graphics processing units, are bypassed, creating a much simpler programming model. In the general purpose parallel computation configuration, the work distribution unit **425** assigns and distributes blocks of threads directly to the processing units within the GPCs **450**. Threads execute the same program, using a unique thread ID in the calculation to ensure each thread generates unique results, using the processing unit(s) to execute the program and perform calculations, shared memory to communicate between threads, and the LSU to read and write global memory through the shared memory and the memory partition unit **480**. When configured for general purpose parallel computation, the processing units can also write commands that the scheduler unit **420** can use to launch new work on the processing units.

The PPUs **400** may each include, and/or be configured to perform functions of, one or more processing cores and/or components thereof, such as Tensor Cores (TCs), Tensor Processing Units (TPUs), Pixel Visual Cores (PVCs), Ray Tracing (RT) Cores, Vision Processing Units (VPUs), Graphics Processing Clusters (GPCs), Texture Processing Clusters (TPCs), Streaming Multiprocessors (SMs), Tree Traversal Units (TTUs), Artificial Intelligence Accelerators (AIAs), Deep Learning Accelerators (DLAs), Arithmetic-Logic Units (ALUs), Application-Specific Integrated Circuits (ASICs), Floating Point Units (FPUs), input/output (I/O) elements, peripheral component interconnect (PCI) or peripheral component interconnect express (PCIe) elements, and/or the like.

The PPU **400** may be included in a desktop computer, a laptop computer, a tablet computer, servers, supercomputers, a smart-phone (e.g., a wireless, hand-held device), personal digital assistant (PDA), a digital camera, a vehicle, a head mounted display, a hand-held electronic device, and the like. In an embodiment, the PPU **400** is embodied on a single semiconductor substrate. In another embodiment, the PPU **400** is included in a system-on-a-chip (SoC) along with one or more other devices such as additional PPUs **400**, the memory **404**, a reduced instruction set computer (RISC) CPU, a memory management unit (MMU), a digital-to-analog converter (DAC), and the like.

In an embodiment, the PPU **400** may be included on a graphics card that includes one or more memory devices. The graphics card may be configured to interface with a PCIe slot on a motherboard of a desktop computer. In yet another embodiment, the PPU **400** may be an integrated graphics processing unit (iGPU) or parallel processor included in the chipset of the motherboard. In yet another embodiment, the PPU **400** may be realized in reconfigurable hardware. In yet another embodiment, parts of the PPU **400** may be realized in reconfigurable hardware.

Exemplary Computing System

Systems with multiple GPUs and CPUs are used in a variety of industries as developers expose and leverage more parallelism in applications such as artificial intelligence computing. High-performance GPU-accelerated systems with tens to many thousands of compute nodes are deployed in data centers, research facilities, and supercomputers to solve ever larger problems. As the number of processing devices within the high-performance systems increases, the communication and data transfer mechanisms need to scale to support the increased bandwidth.

FIG. **5**A is a conceptual diagram of a processing system **500** implemented using the PPU **400** of FIG. **4**, in accordance with an embodiment. The processing system **500** includes a CPU **530**, switch **510**, and multiple PPUs **400**, and respective memories **404**.

The NVLink **410** provides high-speed communication links between each of the PPUs **400**. Although a particular number of NVLink **410** and interconnect **402** connections are illustrated in FIG. **5**B, the number of connections to each PPU **400** and the CPU **530** may vary. The switch **510** interfaces between the interconnect **402** and the CPU **530**. The PPUs **400**, memories **404**, and NVLinks **410** may be situated on a single semiconductor platform to form a parallel processing module **525**. In an embodiment, the switch **510** supports two or more protocols to interface between various different connections and/or links.

In another embodiment (not shown), the NVLink **410** provides one or more high-speed communication links between each of the PPUs **400** and the CPU **530** and the switch **510** interfaces between the interconnect **402** and each of the PPUs **400**. The PPUs **400**, memories **404**, and interconnect **402** may be situated on a single semiconductor platform to form a parallel processing module **525**. In yet another embodiment (not shown), the interconnect **402** provides one or more communication links between each of the PPUs **400** and the CPU **530** and the switch **510** interfaces between each of the PPUs **400** using the NVLink **410** to provide one or more high-speed communication links between the PPUs **400**. In another embodiment (not shown), the NVLink **410** provides one or more high-speed communication links between the PPUs **400** and the CPU **530** through the switch **510**. In yet another embodiment (not shown), the interconnect **402** provides one or more communication links between each of the PPUs **400** directly. One or more of the NVLink **410** high-speed communication links may be implemented as a physical NVLink interconnect or either an on-chip or on-die interconnect using the same protocol as the NVLink **410**.

In the context of the present description, a single semiconductor platform may refer to a sole unitary semiconductor-based integrated circuit fabricated on a die or chip. It should be noted that the term single semiconductor platform may also refer to multi-chip modules with increased connectivity which simulate on-chip operation and make substantial improvements over utilizing a conventional bus implementation. Of course, the various circuits or devices may also be situated separately or in various combinations of semiconductor platforms per the desires of the user. Alternately, the parallel processing module **525** may be implemented as a circuit board substrate and each of the PPUs **400** and/or memories **404** may be packaged devices. In an embodiment, the CPU **530**, switch **510**, and the parallel processing module **525** are situated on a single semiconductor platform.

In an embodiment, the signaling rate of each NVLink **410** is 20 to 25 Gigabits/second and each PPU **400** includes six NVLink **410** interfaces (as shown in FIG. **5**A, five NVLink **410** interfaces are included for each PPU **400**). Each NVLink **410** provides a data transfer rate of 25 Gigabytes/second in each direction, with six links providing 400 Gigabytes/second. The NVLinks **410** can be used exclusively for PPU-to-PPU communication as shown in FIG. **5**A, or some combination of PPU-to-PPU and PPU-to-CPU, when the CPU **530** also includes one or more NVLink **410** interfaces.

In an embodiment, the NVLink **410** allows direct load/store/atomic access from the CPU **530** to each PPU's **400** memory **404**. In an embodiment, the NVLink **410** supports coherency operations, allowing data read from the memories **404** to be stored in the cache hierarchy of the CPU **530**, reducing cache access latency for the CPU **530**. In an embodiment, the NVLink **410** includes support for Address Translation Services (ATS), allowing the PPU **400** to directly access page tables within the CPU **530**. One or more of the NVLinks **410** may also be configured to operate in a low-power mode.

FIG. **5**B illustrates an exemplary system **565** in which the various architecture and/or functionality of the various previous embodiments may be implemented. As shown, a system **565** is provided including at least one central processing unit **530** that is connected to a communication bus **575**. The communication bus **575** may directly or indirectly couple one or more of the following devices: main memory **540**, network interface **535**, CPU(s) **530**, display device(s) **545**, input device(s) **560**, switch **510**, and parallel processing system **525**. The communication bus **575** may be implemented using any suitable protocol and may represent one or more links or busses, such as an address bus, a data bus, a control bus, or a combination thereof. The communication bus **575** may include one or more bus or link types, such as an industry standard architecture (ISA) bus, an extended industry standard architecture (EISA) bus, a video electronics standards association (VESA) bus, a peripheral component interconnect (PCI) bus, a peripheral component interconnect express (PCIe) bus, HyperTransport, and/or another type of bus or link. In some embodiments, there are direct connections between components. As an example, the CPU(s) **530** may be directly connected to the main memory **540**. Further, the CPU(s) **530** may be directly connected to the parallel processing system **525**. Where there is direct, or point-to-point connection between components, the communication bus **575** may include a PCIe link to carry out the connection. In these examples, a PCI bus need not be included in the system **565**.

Although the various blocks of FIG. **5**C are shown as connected via the communication bus **575** with lines, this is not intended to be limiting and is for clarity only. For example, in some embodiments, a presentation component, such as display device(s) **545**, may be considered an I/O component, such as input device(s) **560** (e.g., if the display is a touch screen). As another example, the CPU(s) **530** and/or parallel processing system **525** may include memory (e.g., the main memory **540** may be representative of a storage device in addition to the parallel processing system **525**, the CPUs **530**, and/or other components). In other words, the computing device of FIG. **5**C is merely illustrative. Distinction is not made between such categories as "workstation," "server," "laptop," "desktop," "tablet," "client device," "mobile device," "hand-held device," "game console," "electronic control unit (ECU)," "virtual reality system," and/or other device or system types, as all are contemplated within the scope of the computing device of FIG. **5**C.

The system **565** also includes a main memory **540**. Control logic (software) and data are stored in the main memory **540** which may take the form of a variety of computer-readable media. The computer-readable media may be any available media that may be accessed by the system **565**. The computer-readable media may include both volatile and nonvolatile media, and removable and non-removable media. By way of example, and not limitation, the computer-readable media may comprise computer-storage media and communication media.

The computer-storage media may include both volatile and nonvolatile media and/or removable and non-removable media implemented in any method or technology for storage of information such as computer-readable instructions, data structures, program modules, and/or other data types. For example, the main memory **540** may store computer-readable instructions (e.g., that represent a program(s) and/or a program element(s), such as an operating system. Computer-storage media may include, but is not limited to, RAM, ROM, EEPROM, flash memory or other memory technology, CD-ROM, digital versatile disks (DVD) or other optical disk storage, magnetic cassettes, magnetic tape, magnetic disk storage or other magnetic storage devices, or any other medium which may be used to store the desired information and which may be accessed by system **565**. As used herein, computer storage media does not comprise signals per se.

The computer storage media may embody computer-readable instructions, data structures, program modules, and/or other data types in a modulated data signal such as a carrier wave or other transport mechanism and includes any information delivery media. The term "modulated data signal" may refer to a signal that has one or more of its characteristics set or changed in such a manner as to encode information in the signal. By way of example, and not limitation, the computer storage media may include wired media such as a wired network or direct-wired connection, and wireless media such as acoustic, RF, infrared and other wireless media. Combinations of any of the above should also be included within the scope of computer-readable media.

Computer programs, when executed, enable the system **565** to perform various functions. The CPU(s) **530** may be configured to execute at least some of the computer-readable instructions to control one or more components of the system **565** to perform one or more of the methods and/or processes described herein. The CPU(s) **530** may each include one or more cores (e.g., one, two, four, eight, twenty-eight, seventy-two, etc.) that are capable of handling a multitude of software threads simultaneously. The CPU(s) **530** may include any type of processor, and may include different types of processors depending on the type of system **565** implemented (e.g., processors with fewer cores for mobile devices and processors with more cores for servers). For example, depending on the type of system **565**, the processor may be an Advanced RISC Machines (ARM) processor implemented using Reduced Instruction Set Computing (RISC) or an x86 processor implemented using Complex Instruction Set Computing (CISC). The system **565** may include one or more CPUs **530** in addition to one or more microprocessors or supplementary co-processors, such as math co-processors.

In addition to or alternatively from the CPU(s) **530**, the parallel processing module **525** may be configured to execute at least some of the computer-readable instructions

to control one or more components of the system **565** to perform one or more of the methods and/or processes described herein. The parallel processing module **525** may be used by the system **565** to render graphics (e.g., 3D graphics) or perform general purpose computations. For example, the parallel processing module **525** may be used for General-Purpose computing on GPUs (GPGPU). In embodiments, the CPU(s) **530** and/or the parallel processing module **525** may discretely or jointly perform any combination of the methods, processes and/or portions thereof.

The system **565** also includes input device(s) **560**, the parallel processing system **525**, and display device(s) **545**. The display device(s) **545** may include a display (e.g., a monitor, a touch screen, a television screen, a heads-up-display (HUD), other display types, or a combination thereof), speakers, and/or other presentation components. The display device(s) **545** may receive data from other components (e.g., the parallel processing system **525**, the CPU(s) **530**, etc.), and output the data (e.g., as an image, video, sound, etc.).

The network interface **535** may enable the system **565** to be logically coupled to other devices including the input devices **560**, the display device(s) **545**, and/or other components, some of which may be built in to (e.g., integrated in) the system **565**. Illustrative input devices **560** include a microphone, mouse, keyboard, joystick, game pad, game controller, satellite dish, scanner, printer, wireless device, etc. The input devices **560** may provide a natural user interface (NUI) that processes air gestures, voice, or other physiological inputs generated by a user. In some instances, inputs may be transmitted to an appropriate network element for further processing. An NUI may implement any combination of speech recognition, stylus recognition, facial recognition, biometric recognition, gesture recognition both on screen and adjacent to the screen, air gestures, head and eye tracking, and touch recognition (as described in more detail below) associated with a display of the system **565**. The system **565** may be include depth cameras, such as stereoscopic camera systems, infrared camera systems, RGB camera systems, touchscreen technology, and combinations of these, for gesture detection and recognition. Additionally, the system **565** may include accelerometers or gyroscopes (e.g., as part of an inertia measurement unit (IMU)) that enable detection of motion. In some examples, the output of the accelerometers or gyroscopes may be used by the system **565** to render immersive augmented reality or virtual reality.

Further, the system **565** may be coupled to a network (e.g., a telecommunications network, local area network (LAN), wireless network, wide area network (WAN) such as the Internet, peer-to-peer network, cable network, or the like) through a network interface **535** for communication purposes. The system **565** may be included within a distributed network and/or cloud computing environment.

The network interface **535** may include one or more receivers, transmitters, and/or transceivers that enable the system **565** to communicate with other computing devices via an electronic communication network, included wired and/or wireless communications. The network interface **535** may include components and functionality to enable communication over any of a number of different networks, such as wireless networks (e.g., Wi-Fi, Z-Wave, Bluetooth, Bluetooth LE, ZigBee, etc.), wired networks (e.g., communicating over Ethernet or InfiniBand), low-power wide-area networks (e.g., LoRaWAN, SigFox, etc.), and/or the Internet.

The system **565** may also include a secondary storage (not shown). The secondary storage includes, for example, a hard disk drive and/or a removable storage drive, representing a floppy disk drive, a magnetic tape drive, a compact disk drive, digital versatile disk (DVD) drive, recording device, universal serial bus (USB) flash memory. The removable storage drive reads from and/or writes to a removable storage unit in a well-known manner. The system **565** may also include a hard-wired power supply, a battery power supply, or a combination thereof (not shown). The power supply may provide power to the system **565** to enable the components of the system **565** to operate.

Each of the foregoing modules and/or devices may even be situated on a single semiconductor platform to form the system **565**. Alternately, the various modules may also be situated separately or in various combinations of semiconductor platforms per the desires of the user. While various embodiments have been described above, it should be understood that they have been presented by way of example only, and not limitation. Thus, the breadth and scope of a preferred embodiment should not be limited by any of the above-described exemplary embodiments, but should be defined only in accordance with the following claims and their equivalents.

Example Network Environments

Network environments suitable for use in implementing embodiments of the disclosure may include one or more client devices, servers, network attached storage (NAS), other backend devices, and/or other device types. The client devices, servers, and/or other device types (e.g., each device) may be implemented on one or more instances of the processing system **500** of FIG. **5**A and/or exemplary system **565** of FIG. **5**B—e.g., each device may include similar components, features, and/or functionality of the processing system **500** and/or exemplary system **565**.

Components of a network environment may communicate with each other via a network(s), which may be wired, wireless, or both. The network may include multiple networks, or a network of networks. By way of example, the network may include one or more Wide Area Networks (WANs), one or more Local Area Networks (LANs), one or more public networks such as the Internet and/or a public switched telephone network (PSTN), and/or one or more private networks. Where the network includes a wireless telecommunications network, components such as a base station, a communications tower, or even access points (as well as other components) may provide wireless connectivity.

Compatible network environments may include one or more peer-to-peer network environments—in which case a server may not be included in a network environment—and one or more client-server network environments—in which case one or more servers may be included in a network environment. In peer-to-peer network environments, functionality described herein with respect to a server(s) may be implemented on any number of client devices.

In at least one embodiment, a network environment may include one or more cloud-based network environments, a distributed computing environment, a combination thereof, etc. A cloud-based network environment may include a framework layer, a job scheduler, a resource manager, and a distributed file system implemented on one or more of servers, which may include one or more core network servers and/or edge servers. A framework layer may include a framework to support software of a software layer and/or one or more application(s) of an application layer. The software or application(s) may respectively include web-based service software or applications. In embodiments, one or more of the client devices may use the web-based service

software or applications (e.g., by accessing the service software and/or applications via one or more application programming interfaces (APIs)). The framework layer may be, but is not limited to, a type of free and open-source software web application framework such as that may use a distributed file system for large-scale data processing (e.g., "big data").

A cloud-based network environment may provide cloud computing and/or cloud storage that carries out any combination of computing and/or data storage functions described herein (or one or more portions thereof). Any of these various functions may be distributed over multiple locations from central or core servers (e.g., of one or more data centers that may be distributed across a state, a region, a country, the globe, etc.). If a connection to a user (e.g., a client device) is relatively close to an edge server(s), a core server(s) may designate at least a portion of the functionality to the edge server(s). A cloud-based network environment may be private (e.g., limited to a single organization), may be public (e.g., available to many organizations), and/or a combination thereof (e.g., a hybrid cloud environment).

The client device(s) may include at least some of the components, features, and functionality of the example processing system **500** of FIG. **5**B and/or exemplary system **565** of FIG. **5**C. By way of example and not limitation, a client device may be embodied as a Personal Computer (PC), a laptop computer, a mobile device, a smartphone, a tablet computer, a smart watch, a wearable computer, a Personal Digital Assistant (PDA), an MP3 player, a virtual reality headset, a Global Positioning System (GPS) or device, a video player, a video camera, a surveillance device or system, a vehicle, a boat, a flying vessel, a virtual machine, a drone, a robot, a handheld communications device, a hospital device, a gaming device or system, an entertainment system, a vehicle computer system, an embedded system controller, a remote control, an appliance, a consumer electronic device, a workstation, an edge device, any combination of these delineated devices, or any other suitable device.

Machine Learning

Deep neural networks (DNNs) developed on processors, such as the PPU **400** have been used for diverse use cases, from self-driving cars to faster drug development, from automatic image captioning in online image databases to smart real-time language translation in video chat applications. Deep learning is a technique that models the neural learning process of the human brain, continually learning, continually getting smarter, and delivering more accurate results more quickly over time. A child is initially taught by an adult to correctly identify and classify various shapes, eventually being able to identify shapes without any coaching. Similarly, a deep learning or neural learning system needs to be trained in object recognition and classification for it get smarter and more efficient at identifying basic objects, occluded objects, etc., while also assigning context to objects.

At the simplest level, neurons in the human brain look at various inputs that are received, importance levels are assigned to each of these inputs, and output is passed on to other neurons to act upon. An artificial neuron or perceptron is the most basic model of a neural network. In one example, a perceptron may receive one or more inputs that represent various features of an object that the perceptron is being trained to recognize and classify, and each of these features is assigned a certain weight based on the importance of that feature in defining the shape of an object.

A deep neural network (DNN) model includes multiple layers of many connected nodes (e.g., perceptrons, Boltzmann machines, radial basis functions, convolutional layers, etc.) that can be trained with enormous amounts of input data to quickly solve complex problems with high accuracy. In one example, a first layer of the DNN model breaks down an input image of an automobile into various sections and looks for basic patterns such as lines and angles. The second layer assembles the lines to look for higher level patterns such as wheels, windshields, and mirrors. The next layer identifies the type of vehicle, and the final few layers generate a label for the input image, identifying the model of a specific automobile brand.

Once the DNN is trained, the DNN can be deployed and used to identify and classify objects or patterns in a process known as inference. Examples of inference (the process through which a DNN extracts useful information from a given input) include identifying handwritten numbers on checks deposited into ATM machines, identifying images of friends in photos, delivering movie recommendations to over fifty million users, identifying and classifying different types of automobiles, pedestrians, and road hazards in driverless cars, or translating human speech in real-time.

During training, data flows through the DNN in a forward propagation phase until a prediction is produced that indicates a label corresponding to the input. If the neural network does not correctly label the input, then errors between the correct label and the predicted label are analyzed, and the weights are adjusted for each feature during a backward propagation phase until the DNN correctly labels the input and other inputs in a training dataset. Training complex neural networks requires massive amounts of parallel computing performance, including floating-point multiplications and additions that are supported by the PPU **400**. Inferencing is less compute-intensive than training, being a latency-sensitive process where a trained neural network is applied to new inputs it has not seen before to classify images, detect emotions, identify recommendations, recognize and translate speech, and generally infer new information.

Neural networks rely heavily on matrix math operations, and complex multi-layered networks require tremendous amounts of floating-point performance and bandwidth for both efficiency and speed. With thousands of processing cores, optimized for matrix math operations, and delivering tens to hundreds of TFLOPS of performance, the PPU **400** is a computing platform capable of delivering performance required for deep neural network-based artificial intelligence and machine learning applications.

Furthermore, images generated applying one or more of the techniques disclosed herein may be used to train, test, or certify DNNs used to recognize objects and environments in the real world. Such images may include scenes of roadways, factories, buildings, urban settings, rural settings, humans, animals, and any other physical object or real-world setting. Such images may be used to train, test, or certify DNNs that are employed in machines or robots to manipulate, handle, or modify physical objects in the real world. Furthermore, such images may be used to train, test, or certify DNNs that are employed in autonomous vehicles to navigate and move the vehicles through the real world. Additionally, images generated applying one or more of the techniques disclosed herein may be used to convey information to users of such machines, robots, and vehicles.

FIG. **5**C illustrates components of an exemplary system **555** that can be used to train and utilize machine learning, in accordance with at least one embodiment. As will be dis-

cussed, various components can be provided by various combinations of computing devices and resources, or a single computing system, which may be under control of a single entity or multiple entities. Further, aspects may be triggered, initiated, or requested by different entities. In at least one embodiment training of a neural network might be instructed by a provider associated with provider environment **506**, while in at least one embodiment training might be requested by a customer or other user having access to a provider environment through a client device **502** or other such resource. In at least one embodiment, training data (or data to be analyzed by a trained neural network) can be provided by a provider, a user, or a third party content provider **524**. In at least one embodiment, client device **502** may be a vehicle or object that is to be navigated on behalf of a user, for example, which can submit requests and/or receive instructions that assist in navigation of a device.

In at least one embodiment, requests are able to be submitted across at least one network **504** to be received by a provider environment **506**. In at least one embodiment, a client device may be any appropriate electronic and/or computing devices enabling a user to generate and send such requests, such as, but not limited to, desktop computers, notebook computers, computer servers, smartphones, tablet computers, gaming consoles (portable or otherwise), computer processors, computing logic, and set-top boxes. Network(s) **504** can include any appropriate network for transmitting a request or other such data, as may include Internet, an intranet, an Ethernet, a cellular network, a local area network (LAN), a wide area network (WAN), a personal area network (PAN), an ad hoc network of direct wireless connections among peers, and so on.

In at least one embodiment, requests can be received at an interface layer **508**, which can forward data to a training and inference manager **532**, in this example. The training and inference manager **532** can be a system or service including hardware and software for managing requests and service corresponding data or content, in at least one embodiment, the training and inference manager **532** can receive a request to train a neural network, and can provide data for a request to a training module **512**. In at least one embodiment, training module **512** can select an appropriate model or neural network to be used, if not specified by the request, and can train a model using relevant training data. In at least one embodiment, training data can be a batch of data stored in a training data repository **514**, received from client device **502**, or obtained from a third party provider **524**. In at least one embodiment, training module **512** can be responsible for training data. A neural network can be any appropriate network, such as a recurrent neural network (RNN) or convolutional neural network (CNN). Once a neural network is trained and successfully evaluated, a trained neural network can be stored in a model repository **516**, for example, that may store different models or networks for users, applications, or services, etc. In at least one embodiment, there may be multiple models for a single application or entity, as may be utilized based on a number of different factors.

In at least one embodiment, at a subsequent point in time, a request may be received from client device **502** (or another such device) for content (e.g., path determinations) or data that is at least partially determined or impacted by a trained neural network. This request can include, for example, input data to be processed using a neural network to obtain one or more inferences or other output values, classifications, or predictions, or for at least one embodiment, input data can be received by interface layer **508** and directed to inference module **518**, although a different system or service can be used as well. In at least one embodiment, inference module **518** can obtain an appropriate trained network, such as a trained deep neural network (DNN) as discussed herein, from model repository **516** if not already stored locally to inference module **518**. Inference module **518** can provide data as input to a trained network, which can then generate one or more inferences as output. This may include, for example, a classification of an instance of input data. In at least one embodiment, inferences can then be transmitted to client device **502** for display or other communication to a user. In at least one embodiment, context data for a user may also be stored to a user context data repository **522**, which may include data about a user which may be useful as input to a network in generating inferences, or determining data to return to a user after obtaining instances. In at least one embodiment, relevant data, which may include at least some of input or inference data, may also be stored to a local database **534** for processing future requests. In at least one embodiment, a user can use account information or other information to access resources or functionality of a provider environment. In at least one embodiment, if permitted and available, user data may also be collected and used to further train models, in order to provide more accurate inferences for future requests. In at least one embodiment, requests may be received through a user interface to a machine learning application **526** executing on client device **502**, and results displayed through a same interface. A client device can include resources such as a processor **528** and memory **562** for generating a request and processing results or a response, as well as at least one data storage element **552** for storing data for machine learning application **526**.

In at least one embodiment a processor **528** (or a processor of training module **512** or inference module **518**) will be a central processing unit (CPU). As mentioned, however, resources in such environments can utilize GPUs to process data for at least certain types of requests. With thousands of cores, GPUs, such as PPU **400** are designed to handle substantial parallel workloads and, therefore, have become popular in deep learning for training neural networks and generating predictions. While use of GPUs for offline builds has enabled faster training of larger and more complex models, generating predictions offline implies that either request-time input features cannot be used or predictions must be generated for all permutations of features and stored in a lookup table to serve real-time requests. If a deep learning framework supports a CPU-mode and a model is small and simple enough to perform a feed-forward on a CPU with a reasonable latency, then a service on a CPU instance could host a model. In this case, training can be done offline on a GPU and inference done in real-time on a CPU. If a CPU approach is not viable, then a service can run on a GPU instance. Because GPUs have different performance and cost characteristics than CPUs, however, running a service that offloads a runtime algorithm to a GPU can require it to be designed differently from a CPU based service.

In at least one embodiment, video data can be provided from client device **502** for enhancement in provider environment **506**. In at least one embodiment, video data can be processed for enhancement on client device **502**. In at least one embodiment, video data may be streamed from a third party content provider **524** and enhanced by third party content provider **524**, provider environment **506**, or client device **502**. In at least one embodiment, video data can be provided from client device **502** for use as training data in provider environment **506**.

In at least one embodiment, supervised and/or unsupervised training can be performed by the client device **502** and/or the provider environment **506**. In at least one embodiment, a set of training data **514** (e.g., classified or labeled data) is provided as input to function as training data. In at least one embodiment, training data can include instances of at least one type of object for which a neural network is to be trained, as well as information that identifies that type of object. In at least one embodiment, training data might include a set of images that each includes a representation of a type of object, where each image also includes, or is associated with, a label, metadata, classification, or other piece of information identifying a type of object represented in a respective image. Various other types of data may be used as training data as well, as may include text data, audio data, video data, and so on. In at least one embodiment, training data **514** is provided as training input to a training module **512**. In at least one embodiment, training module **512** can be a system or service that includes hardware and software, such as one or more computing devices executing a training application, for training a neural network (or other model or algorithm, etc.). In at least one embodiment, training module **512** receives an instruction or request indicating a type of model to be used for training, in at least one embodiment, a model can be any appropriate statistical model, network, or algorithm useful for such purposes, as may include an artificial neural network, deep learning algorithm, learning classifier, Bayesian network, and so on. In at least one embodiment, training module **512** can select an initial model, or other untrained model, from an appropriate repository **516** and utilize training data **514** to train a model, thereby generating a trained model (e.g., trained deep neural network) that can be used to classify similar types of data, or generate other such inferences. In at least one embodiment where training data is not used, an appropriate initial model can still be selected for training on input data per training module **512**.

In at least one embodiment, a model can be trained in a number of different ways, as may depend in part upon a type of model selected. In at least one embodiment, a machine learning algorithm can be provided with a set of training data, where a model is a model artifact created by a training process. In at least one embodiment, each instance of training data contains a correct answer (e.g., classification), which can be referred to as a target or target attribute. In at least one embodiment, a learning algorithm finds patterns in training data that map input data attributes to a target, an answer to be predicted, and a machine learning model is output that captures these patterns. In at least one embodiment, a machine learning model can then be used to obtain predictions on new data for which a target is not specified.

In at least one embodiment, training and inference manager **532** can select from a set of machine learning models including binary classification, multiclass classification, generative, and regression models. In at least one embodiment, a type of model to be used can depend at least in part upon a type of target to be predicted.

Graphics Processing Pipeline

In an embodiment, the PPU **400** comprises a graphics processing unit (GPU). The PPU **400** is configured to receive commands that specify shader programs for processing graphics data. Graphics data may be defined as a set of primitives such as points, lines, triangles, quads, triangle strips, and the like. Typically, a primitive includes data that specifies a number of vertices for the primitive (e.g., in a model-space coordinate system) as well as attributes associated with each vertex of the primitive. The PPU **400** can be configured to process the graphics primitives to generate a frame buffer (e.g., pixel data for each of the pixels of the display).

An application writes model data for a scene (e.g., a collection of vertices and attributes) to a memory such as a system memory or memory **404**. The model data defines each of the objects that may be visible on a display. The application then makes an API call to the driver kernel that requests the model data to be rendered and displayed. The driver kernel reads the model data and writes commands to the one or more streams to perform operations to process the model data. The commands may reference different shader programs to be implemented on the processing units within the PPU **400** including one or more of a vertex shader, hull shader, domain shader, geometry shader, and a pixel shader. For example, one or more of the processing units may be configured to execute a vertex shader program that processes a number of vertices defined by the model data. In an embodiment, the different processing units may be configured to execute different shader programs concurrently. For example, a first subset of processing units may be configured to execute a vertex shader program while a second subset of processing units may be configured to execute a pixel shader program. The first subset of processing units processes vertex data to produce processed vertex data and writes the processed vertex data to the L2 cache **460** and/or the memory **404**. After the processed vertex data is rasterized (e.g., transformed from three-dimensional data into two-dimensional data in screen space) to produce fragment data, the second subset of processing units executes a pixel shader to produce processed fragment data, which is then blended with other processed fragment data and written to the frame buffer in memory **404**. The vertex shader program and pixel shader program may execute concurrently, processing different data from the same scene in a pipelined fashion until all of the model data for the scene has been rendered to the frame buffer. Then, the contents of the frame buffer are transmitted to a display controller for display on a display device.

Images generated applying one or more of the techniques disclosed herein may be displayed on a monitor or other display device. In some embodiments, the display device may be coupled directly to the system or processor generating or rendering the images. In other embodiments, the display device may be coupled indirectly to the system or processor such as via a network. Examples of such networks include the Internet, mobile telecommunications networks, a WIFI network, as well as any other wired and/or wireless networking system. When the display device is indirectly coupled, the images generated by the system or processor may be streamed over the network to the display device. Such streaming allows, for example, video games or other applications, which render images, to be executed on a server, a data center, or in a cloud-based computing environment and the rendered images to be transmitted and displayed on one or more user devices (such as a computer, video game console, smartphone, other mobile device, etc.) that are physically separate from the server or data center. Hence, the techniques disclosed herein can be applied to enhance the images that are streamed and to enhance services that stream images such as NVIDIA GeForce Now (GFN), Google Stadia, and the like.

Example Streaming System

FIG. **6** is an example system diagram for a streaming system **605**, in accordance with some embodiments of the present disclosure. FIG. **6** includes server(s) **603** (which may include similar components, features, and/or functionality to

the example processing system **500** of FIG. **5**A and/or exemplary system **565** of FIG. **5**B), client device(s) **604** (which may include similar components, features, and/or functionality to the example processing system **500** of FIG. **5**A and/or exemplary system **565** of FIG. **5**B), and network(s) **606** (which may be similar to the network(s) described herein). In some embodiments of the present disclosure, the system **605** may be implemented.

In an embodiment, the streaming system **605** is a game streaming system and the sever(s) **604** are game server(s). In the system **605**, for a game session, the client device(s) **604** may only receive input data in response to inputs to the input device(s) **626**, transmit the input data to the server(s) **603**, receive encoded display data from the server(s) **603**, and display the display data on the display **624**. As such, the more computationally intense computing and processing is offloaded to the server(s) **603** (e.g., rendering—in particular ray or path tracing—for graphical output of the game session is executed by the GPU(s) **615** of the server(s) **603**). In other words, the game session is streamed to the client device(s) **604** from the server(s) **603**, thereby reducing the requirements of the client device(s) **604** for graphics processing and rendering.

For example, with respect to an instantiation of a game session, a client device **604** may be displaying a frame of the game session on the display **624** based on receiving the display data from the server(s) **603**. The client device **604** may receive an input to one of the input device(s) **626** and generate input data in response. The client device **604** may transmit the input data to the server(s) **603** via the communication interface **621** and over the network(s) **606** (e.g., the Internet), and the server(s) **603** may receive the input data via the communication interface **618**. The CPU(s) **608** may receive the input data, process the input data, and transmit data to the GPU(s) **615** that causes the GPU(s) **615** to generate a rendering of the game session. For example, the input data may be representative of a movement of a character of the user in a game, firing a weapon, reloading, passing a ball, turning a vehicle, etc. The rendering component **612** may render the game session (e.g., representative of the result of the input data) and the render capture component **614** may capture the rendering of the game session as display data (e.g., as image data capturing the rendered frame of the game session). The rendering of the game session may include ray or path-traced lighting and/or shadow effects, computed using one or more parallel processing units—such as GPUs, which may further employ the use of one or more dedicated hardware accelerators or processing cores to perform ray or path-tracing techniques—of the server(s) **603**. The encoder **616** may then encode the display data to generate encoded display data and the encoded display data may be transmitted to the client device **604** over the network(s) **606** via the communication interface **618**. The client device **604** may receive the encoded display data via the communication interface **621** and the decoder **622** may decode the encoded display data to generate the display data. The client device **604** may then display the display data via the display **624**.

It is noted that the techniques described herein may be embodied in executable instructions stored in a computer readable medium for use by or in connection with a processor-based instruction execution machine, system, apparatus, or device. It will be appreciated by those skilled in the art that, for some embodiments, various types of computer-readable media can be included for storing data. As used herein, a "computer-readable medium" includes one or more of any suitable media for storing the executable instructions of a computer program such that the instruction execution machine, system, apparatus, or device may read (or fetch) the instructions from the computer-readable medium and execute the instructions for carrying out the described embodiments. Suitable storage formats include one or more of an electronic, magnetic, optical, and electromagnetic format. A non-exhaustive list of conventional exemplary computer-readable medium includes: a portable computer diskette; a random-access memory (RAM); a read-only memory (ROM); an erasable programmable read only memory (EPROM); a flash memory device; and optical storage devices, including a portable compact disc (CD), a portable digital video disc (DVD), and the like.

It should be understood that the arrangement of components illustrated in the attached Figures are for illustrative purposes and that other arrangements are possible. For example, one or more of the elements described herein may be realized, in whole or in part, as an electronic hardware component. Other elements may be implemented in software, hardware, or a combination of software and hardware. Moreover, some or all of these other elements may be combined, some may be omitted altogether, and additional components may be added while still achieving the functionality described herein. Thus, the subject matter described herein may be embodied in many different variations, and all such variations are contemplated to be within the scope of the claims.

To facilitate an understanding of the subject matter described herein, many aspects are described in terms of sequences of actions. It will be recognized by those skilled in the art that the various actions may be performed by specialized circuits or circuitry, by program instructions being executed by one or more processors, or by a combination of both. The description herein of any sequence of actions is not intended to imply that the specific order described for performing that sequence must be followed. All methods described herein may be performed in any suitable order unless otherwise indicated herein or otherwise clearly contradicted by context.

The use of the terms "a" and "an" and "the" and similar references in the context of describing the subject matter (particularly in the context of the following claims) are to be construed to cover both the singular and the plural, unless otherwise indicated herein or clearly contradicted by context. The use of the term "at least one" followed by a list of one or more items (for example, "at least one of A and B") is to be construed to mean one item selected from the listed items (A or B) or any combination of two or more of the listed items (A and B), unless otherwise indicated herein or clearly contradicted by context. Furthermore, the foregoing description is for the purpose of illustration only, and not for the purpose of limitation, as the scope of protection sought is defined by the claims as set forth hereinafter together with any equivalents thereof. The use of any and all examples, or exemplary language (e.g., "such as") provided herein, is intended merely to better illustrate the subject matter and does not pose a limitation on the scope of the subject matter unless otherwise claimed. The use of the term "based on" and other like phrases indicating a condition for bringing about a result, both in the claims and in the written description, is not intended to foreclose any other conditions that bring about that result. No language in the specification should be construed as indicating any non-claimed element as essential to the practice of the invention as claimed.

What is claimed is:

1. A computer-implemented method, comprising:

processing, according to an optical model for lithography, a mask image defining shapes for fabrication of an integrated circuit by a first branch of a neural network to extract low-frequency component features by converting the mask image into a frequency domain and performing at least a linear operation in the frequency domain;

extracting high-frequency component features from the mask image by a second branch of the neural network; and

producing an estimated fabrication image by processing the low-frequency component features and the high-frequency component features by a reconstruction portion of the neural network.

2. The computer-implemented method of claim 1, wherein the mask image is downsampled for processing by the first branch.

3. The computer-implemented method of claim 2, further comprising subdividing the downsampled mask image into tiles for processing by the first branch.

4. The computer-implemented method of claim 3, wherein each one of the tiles at least partially overlaps with an adjacent tile.

5. The computer-implemented method of claim 3, wherein the first branch extracts high-frequency component features for each processed tile and further comprising concatenating the high-frequency component features for the processed tiles for input to the reconstruction portion of the neural network.

6. The computer-implemented method of claim 1, wherein the low-frequency component features correspond to light intensity.

7. The computer-implemented method of claim 1, wherein the high-frequency component features correspond to contour and shape edge details.

8. The computer-implemented method of claim 1, wherein the first branch processes the mask image by:

performing a Fourier transform on the mask image to convert the mask image into the frequency domain;

convolving the converted mask image with a channel-lifting operator;

performing the linear operation on the convolved converted mask image to extract low-frequency features in the frequency domain; and

performing an inverse Fourier transform on the extracted low-frequency features in the frequency domain to produce the low-frequency component features.

9. The computer-implemented method of claim 1, wherein at least one of the steps of processing, extracting, and producing are performed on a server or in a data center and the estimated fabrication image is streamed to a user device.

10. The computer-implemented method of claim 1, wherein at least one of the steps of processing, extracting, and producing are performed within a cloud computing environment.

11. The computer-implemented method of claim 1, wherein the integrated circuit is employed in a machine, robot, or autonomous vehicle.

12. The computer-implemented method of claim 1, wherein at least one of the steps of processing, extracting, and producing is performed on a virtual machine comprising a portion of a graphics processing unit.

13. A computer-implemented method, comprising:

processing a fabrication image defining patterns for an integrated circuit by a first branch of a neural network according to an optical model for lithography to extract low-frequency component features by converting the mask image into a frequency domain and performing at least a linear operation in the frequency domain;

extracting high-frequency component features from the fabrication image by a second branch of the neural network; and

producing an estimated mask image by processing the low-frequency component features and the high-frequency component features by a reconstruction portion of the neural network.

14. The computer-implemented method of claim 13, wherein the low-frequency component features correspond to light intensity.

15. The computer-implemented method of claim 13, wherein the high-frequency component features correspond to contour and shape edge details.

16. A system, comprising:

a memory that stores a mask image defining shapes for fabrication of an integrated circuit; and

a processor that is connected to the memory, wherein the processor implements a neural network comprising:

a first branch that processes the mask image to extract low-frequency component features by converting the mask image into a frequency domain and performing at least a linear operation in the frequency domain;

a second branch that extracts high-frequency component features from the mask image; and

a reconstruction portion that produces an estimated fabrication image by processing the low-frequency component features and the high-frequency component features.

17. The system of claim 16, wherein the mask image is downsampled for processing by the first branch.

18. The system of claim 16, wherein the low-frequency component features correspond to light intensity.

19. A non-transitory computer-readable media storing computer instructions that, when executed by one or more processors, cause the one or more processors to perform the steps of:

processing, according to an optical model for lithography, a mask image defining shapes for fabrication of an integrated circuit by a first branch of a neural network to extract low-frequency component features by converting the mask image into a frequency domain and performing at least a linear operation in the frequency domain;

extracting high-frequency component features from the mask image by a second branch of the neural network; and

producing an estimated fabrication image by processing the low-frequency component features and the high-frequency component features by a reconstruction portion of the neural network.

20. The non-transitory computer-readable media of claim 19, wherein the low-frequency component features correspond to light intensity.

* * * * *